

TP11 : Javascript

SOMMAIRE

Introduction.....	4
Exercice 1 : Afficher une alerte.....	5
Afficher une alerte avec un script externe.....	6
Instruction.....	8
Point-virgule.....	8
Erreur avec les points-virgules.....	9
Les commentaires.....	9
Raccourci.....	10
Le mode moderne “use strict”.....	11
Variable.....	11
Exercice : Travailler avec des variables.....	12
Assigner le bon nom.....	14
Constante en majuscule ?.....	15
Les types de données.....	15
Number.....	16
Infinity.....	16
BigInt.....	16
String.....	17
Boolean.....	17
Null.....	18
La valeur “undefined”.....	18
Exercice : String quotes.....	19
Exercice : Une simple page.....	19
Exercice : Les formes postfixes et préfixes.....	20
Résultat d’affectation.....	20
Les conversions de type.....	21
Corrigez l’addition.....	22
Comparaison.....	23
if (une chaîne de caractères avec zéro).....	24
Afficher le signe.....	29
Réécrire ‘if’ en ‘?’.....	30
Quel est le résultat de OR ?.....	31
Quel est le résultat des alertes OR ?.....	32
Quel est le résultat des alertes AND ?.....	33
Le résultat de OR AND OR.....	34
Vérifiez la plage entre.....	35
Vérifiez à l’extérieur de la plage.....	36
Une question à propos de “if”.....	37

Check the login.....	38
Quelles valeurs sont affichées par la boucle “for” ?	47
Extraire les nombres pairs dans la boucle.....	48
Remplacer “for” par “while”.....	50
Répéter jusqu’à ce que l’entrée soit correcte.....	51
Extraire des nombres premiers.....	52
Réécrire le “if” dans un “switch”	55
Est-ce que “else” est requis ?.....	57
Réécrivez la fonction en utilisant ‘?’ ou ‘ ’.....	59
Fonction min(a,b).....	61
Fonction pow(x,n).....	62
Réécrire avec les fonctions fléchées.....	64
Débogage.....	66

Introduction

JavaScript est aujourd'hui l'un des langages de programmation les plus incontournables du développement web.

Conçu à l'origine pour rendre les pages interactives et vivantes, il s'est rapidement imposé comme un outil universel, capable de fonctionner aussi bien dans les navigateurs que sur les serveurs ou même dans des applications mobiles. Contrairement à d'autres langages, il ne permet pas d'accéder directement à la mémoire ou au processeur, car son objectif initial était de fonctionner dans les navigateurs. Au fil du temps, Javascript a évolué pour devenir un langage indépendant, doté de ses propres spécifications.

Néanmoins, la syntaxe de JavaScript ne convient pas à tous. L'apparition de nombreux **langages transpilés** en Javascript a grandement facilité le travail des développeurs. Ces langages leur permettent de coder avec des fonctionnalités adaptées à leurs besoins, puis d'exécuter le code dans le navigateur.

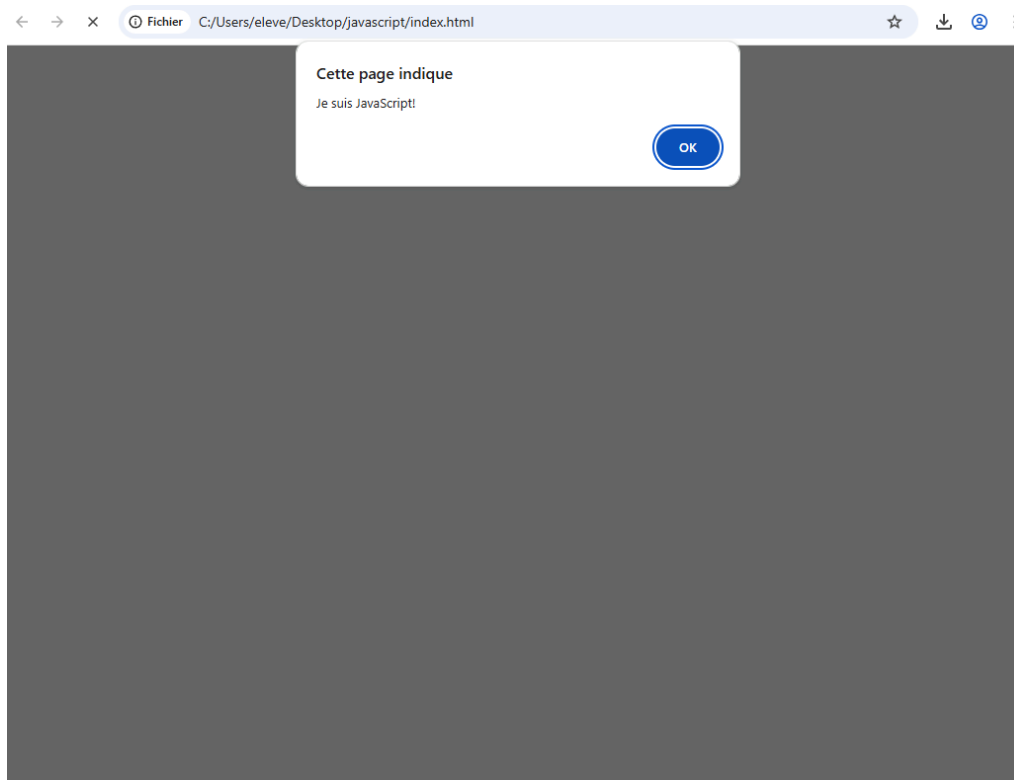
Exercice 1 : Afficher une alerte

Après avoir créer un fichier nommé index.html nous allons mettre nos alerte javascript directement dedans.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
</head>
<body>
  <script>
    alert("Je suis JavaScript!");
  </script>
</body>
</html>
```



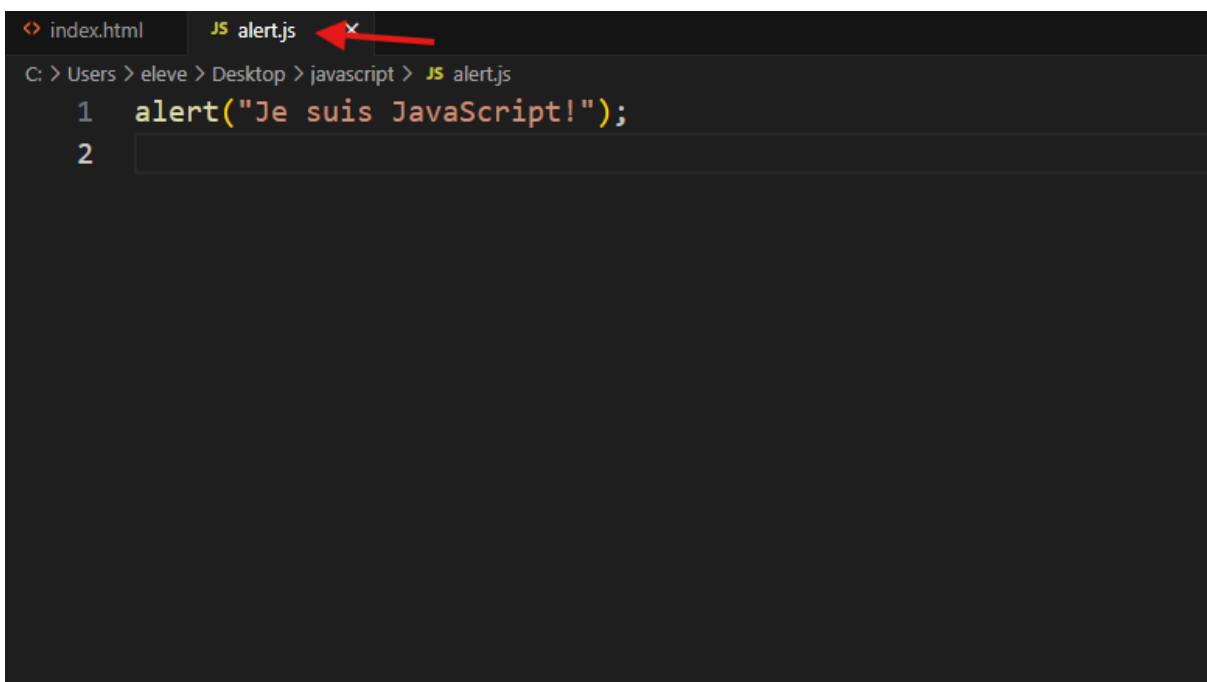
Résultat :



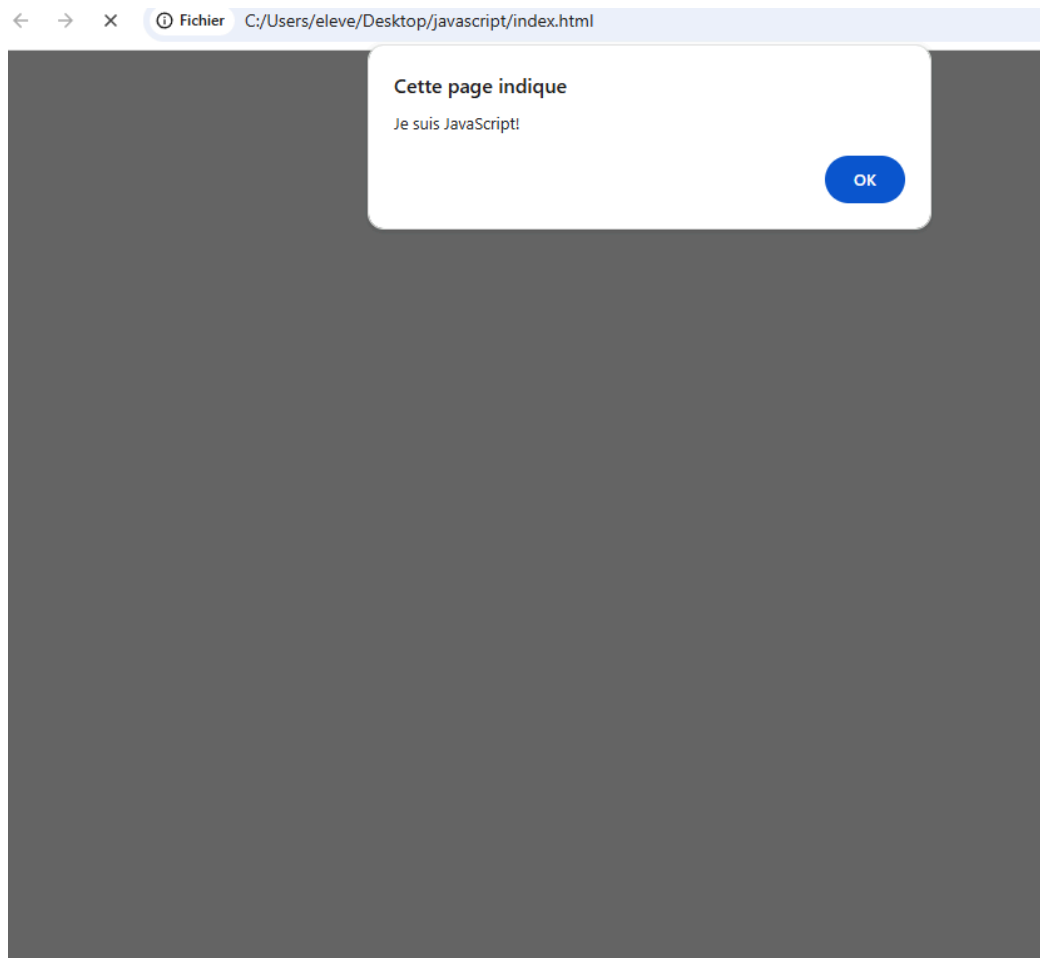
Afficher une alerte avec un script externe

Nous allons à présent créer une alerte avec un script externe, pour cela nous gardons notre fichier html en plus de créer un fichier externe nommé "[alerte.js](#)"

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
</head>
<body>
  <script src="alert.js"></script>
</body>
</html>
```



```
index.html JS alert.js
C: > Users > eleve > Desktop > javascript > JS alert.js
1 alert("Je suis JavaScript!");
2
```



Instruction

En JavaScript, une instruction est une commande qui réalise une action, comme `alert("Hello")`. Les instructions sont généralement écrites sur des lignes séparées et peuvent être terminées par un point-virgule afin d'améliorer la visibilité du code.

Point-virgule

JavaScript peut aussi ajouter automatiquement ce point virgule lorsqu'il détecte un saut de ligne et qu'un point virgule a été omis, on nomme cela "l'insertion automatique de point-virgule".

Cependant cette insertion n'est pas toujours fiable. Si une ligne se termine par une expression incomplète, comme un "+" par exemple, Javascript considère que l'instruction continue et **n'ajoute pas de point virgule**.

Cas concret :

```
1  alert("Je suis JavaScript!");  
2  alert("Hello")  
3  
4  
5  
6  
7
```

Ici nous avons volontairement omis de mettre un point virgule, cependant Javascript a détecté un saut de ligne et a lui-même intégré un point virgule en interne même si celui-ci n'apparaît pas visuellement dans l'éditeur.

Erreur avec les points-virgules

JavaScript peut insérer automatiquement des points-virgules lorsque les instructions sont séparées par des retours à la ligne. Cela fonctionne la plupart du temps, mais il existe des cas où cette insertion automatique échoue. L'exemple ci-dessous montre que si l'on écrit :

```
alert("Je suis JavaScript!");  
alert("Hello")  
[1, 2]. forEach(alert);
```



JavaScript ne voit pas deux instructions séparées. Il fusionne les deux lignes et crée une erreur, car la seconde ligne commence par un crochet ([).

Les commentaires

Dans un programme, les commentaires servent à expliquer le code et à le rendre plus facile à comprendre. Les commentaires sur une ligne commencent par deux barres obliques // et ignore tout ce qui suit.

```
1  alert("Je suis JavaScript!");  
2  alert("Hello")  
3  [1, 2]. forEach(alert);  
4  
5  
6  // commentaire  
7  
8  
9
```



Les commentaires multilignes quant à eux sont utiles pour écrire plusieurs lignes de texte ou désactiver temporairement du code.

```
1  alert("Je suis JavaScript!");
2  alert("Hello")
3  [1, 2]. forEach(alert);
4
5
6  // commentaire
7
8  /* commentaire
9
10
11
12
13
14
15
```



Raccourci

Il existe aussi des raccourcis comme Ctrl + : pour ajouter rapidement un commentaire.



Le mode moderne "use strict"

Le mode "use strict" a été introduit avec ECMAScript 5 pour rendre JavaScript plus strict et sécurisé. Il permet d'éviter des erreurs fréquentes en bloquant certains comportements dangereux ou anciens du langage. Pour activer ce mode, il suffit d'écrire "use strict" au tout début du script ou au début d'une fonction.

```
C: > Users > eleve > Desktop > javascript > JS alert.js
1  "use strict";
2
3  alert("Je suis JavaScript!");
4  alert("Hello")
5  [1, 2]. forEach(alert);
6
7
```

Variable

Une variable est un espace de stockage nommé qui permet de conserver des données. Pour créer une variable en JavaScript, on utilise le mot-clé `let`. On peut d'abord déclarer la variable, puis lui attribuer une valeur grâce à l'opérateur d'affectation `=`.

Une variable sert donc à stocker des données. On la crée avec le mot-clé `let`, par exemple :

```
let message = 'Hello'
```

Exercice : Travailler avec des variables

1. Nous allons déclarer deux variables : "admin" et "name".

```
C: > Users > eleve > Desktop > javascript > JS alert.js > ...  
1  let admin;  
2  let name;  
3  
4
```

2. Nous allons assigner la valeur "John" à name.

```
let admin;  
let name;  
  
name = "John";  
      ↑
```

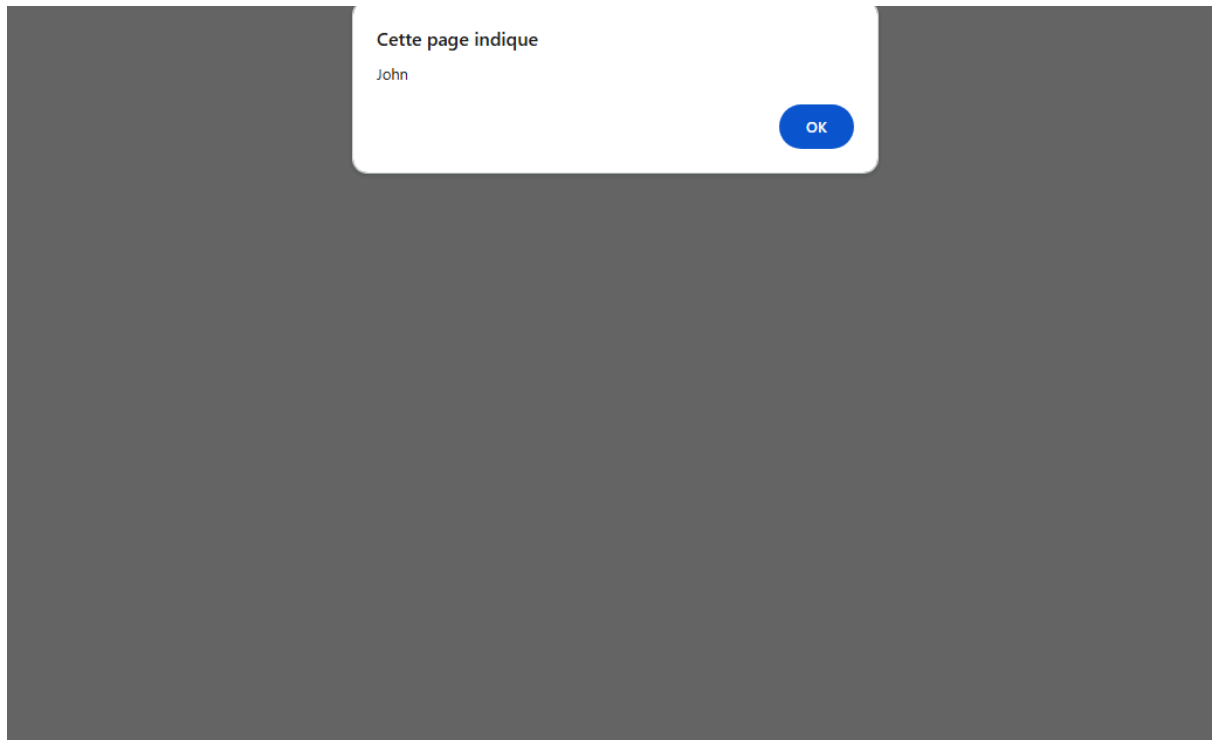
3. Copions la valeur de name à admin

```
C: > Users > eleve > Desktop > javascript > JS alert.js > ...  
1  let admin;  
2  let name;  
3  name = "John";  
4  admin = name;  
5  
6  
7
```

4. Affichons la valeur de admin en utilisant alert

```
1  let admin;  
2  let name;  
3  name = "John";  
4  admin = name;  
5  alert(admin);  
6  
7  
8
```

Résultat :



Assigner le bon nom

1. Nous allons créer la variable avec le nom de notre planète que nous nommons "planetname"

```
1  let admin;  
2  let name;  
3  name = "John";  
4  admin = name;  
5  alert(admin);  
6  let planetname = "Earth";  
7  
8  
9
```



2. Pour stocker le nom de la personne qui utilise actuellement le site, nous avons choisi le nom de variable "CurrentNameUser".

```
1  let admin;  
2  let name;  
3  name = "John";  
4  admin = name;  
5  alert(admin);  
6  let planetname = "Earth";  
7  let currentnameuser = "John";  
8  
9
```

Constante en majuscule ?

```
1 const birthday = '18.04.1982';  
2  
3 const age = someCode(birthday);
```

Dans l'exemple donné, birthday est une date de naissance, donc une valeur inchangeable, nous pouvons l'écrire en majuscules (BIRTHDAY).

En revanche, l'âge est une valeur calculée à partir de cette date: elle peut varier. Nous devons donc la laisser en minuscules.

Serait-il juste d'utiliser des majuscules pour birthday ? Pour âge? Ou même pour les deux ?

Non, il ne serait pas juste de mettre les deux en majuscules car nous utilisons les majuscules uniquement pour les constantes dont la valeur ne changera jamais.

```
1 const BIRTHDAY = '18.04.1982'; // mettre l'anniversaire en majuscule ?  
2  
3 const AGE = someCode(BIRTHDAY); // mettre l'âge en majuscule ?
```

Les types de données

En JavaScript, chaque valeur possède un type. Il existe huit types de données de base, et nous pouvons mettre n'importe quel type dans une variable, par exemple une variable peut contenir une chaîne de caractères puis plus tard un nombre.

```
1 // pas d'erreur  
2 let message = "hello";  
3 message = 123456;
```

C'est pour cette raison que JavaScript est un langage appelé "typé dynamiquement" ce qui signifie qu'il existe des types de données, mais que les variables ne sont liées à aucun d'entre eux.

Number

```
1 let n = 123;  
2 n = 12.345;
```

Le type number sert à représenter aussi bien des nombres entiers que des nombres réels.

Infinity

Infinity représente l'infini. C'est une valeur spéciale qui peut être obtenue en divisant un nombre par zéro :

```
alert( 1 / 0 );
```

Ou en le mentionnant dans le code directement :

```
alert ( Infinity ) ;
```

BigInt

Le type number ne peut pas représenter correctement des entiers trop grands. Au-delà d'une certaine limite, les nombres deviennent approximatifs et perdent de leur précision. Pour éviter ce problème, JavaScript propose un nouveau type : BigInt.

```
const bigint = 123456789012345678901234567890n;
```


String

Une string est une chaîne de caractères entourées de guillemets. En JavaScript, nous pouvons utiliser 3 types de guillemets.

1. Double quotes : "Hello".
2. Single quotes : 'Hello'
3. Backticks: `Hello`.

Les backticks nous permettent d'intégrer des variables ou des expressions directement dans la chaîne grâce à `${ }`.

```
1 let name = "John";  
2  
3 // une variable encapsulée  
4 alert( `Hello, ${name}!` ); // Hello, John!  
5  
6 // une expression encapsulée  
7 alert( `the result is ${1 + 2}` ); // le résultat est 3
```

Boolean

Le type booléen ne possède que deux valeurs : True et false. Nous l'utilisons pour représenter des choix oui/non et pour le résultat d'une comparaison.

```
1 let nameFieldChecked = true; // oui, le champ de nom est coché  
2 let ageFieldChecked = false; // non, le champ d'âge n'est pas coché
```

```
1 let isGreater = 4 > 1;  
2  
3 alert( isGreater ); // true (le résultat de la comparaison est "oui")
```

Null

La valeur null est pour les valeurs inconnues, elle est simplement un type spécial qui indique qu'une variable n'a volontairement aucune valeur.

```
1 let age = null;
```

La valeur “undefined”

La valeur undefined est un type à part entière, comme null. La signification de undefined est la “valeur n'est pas attribuée”.

Si une variable est déclarée mais non affectée, alors sa valeur est exactement undefined.

Exercice : String quotes

Quelle est la sortie du script ?

```
1 let name = "Ilya";
2
3 alert( `hello ${1}` ); // ?
4
5 alert( `hello ${"name"}` ); // ?
6
7 alert( `hello ${name}` ); // ?
```

Le script va afficher hello 1, hello name, et hello Ilya.

Exercice : Une simple page

Nous allons à présent créer une page Web qui demande un nom et l'affiche.

```
<script>
let name = prompt("Quel est ton nom ?");
</script>
```

A screenshot of a web browser window with a dark gray background. In the center, a white prompt dialog box is displayed. The dialog box has a title bar that says "Cette page indique". Below the title bar, the text "Quel est ton nom ?" is shown. There is a text input field below the text. At the bottom of the dialog box, there are two buttons: a blue "OK" button and a light blue "Annuler" button.

Exercice : Les formes postfixes et préfixes

Quelles sont les valeurs finales de toutes les variables a, b, c et d après le code ci-dessous?

```
let a = 1, b = 1;  
let c = ++a; // ?  
let d = b++; // ?
```

Le code commence avec a = 1 et b = 1. Ensuite, c = ++a incrémente a avant de l'utiliser, donc a devient 2 et c vaut 2. Puis d = b++ utilise la valeur de b avant de l'incrémenter, donc d vaut 1 et b devient 2. À la fin, les valeurs sont a = 2, b = 2, c = 2 et d = 1.

Résultat d'affectation

Quelles sont les valeurs de a et x après le code ci-dessous ?

```
let a = 2;  
let x = 1 + (a *= 2);
```

Le code commence avec a = 2. L'instruction a *= 2 multiplie a par 2 et réaffecte le résultat, donc a devient 4. Ensuite x = 1 + (a *= 2) utilise la nouvelle valeur de a, qui est 4, et ajoute 1, ce qui donne x = 5. À la fin de l'exécution, les valeurs finales sont a = 4 et x = 5.

Les conversions de type

Quels sont les résultats de ces expressions ?

```
"" + 1 + 0
"" - 1 + 0
true + false
6 / "3"
"2" * "3"
4 + 5 + "px"
"$" + 4 + 5
"4" - 2
"4px" - 2
" - 9 " + 5
" - 9 " - 5
null + 1
undefined + 1
" \t \n" - 2
```

"" + 1 + 0 = "10"

"" - 1 + 0 = -1

true + false = 1

6 / "3" = 2

"2" * "3" = 6

4 + 5 + "px" = "9px"

"\$" + 4 + 5 = "\$45"

"4" - 2 = 2

"4px" - 2 = NaN

" - 9 " + 5 = " - 9 5"

" - 9 " - 5 = -14

null + 1 → 1

undefined + 1 = NaN

" \t \n" - 2 = -2

Corrigez l'addition

Le problème vient du fait que prompt renvoie des chaînes de caractères. L'opérateur + concatène donc les strings ("1" + "2" = "12"). Il faut donc convertir les entrées en nombres et corriger la syntaxe.

```
let a = Number(prompt("First number?", 1));  
let b = Number(prompt("Second number?", 2));  
alert(a + b); // 3
```

```
let a = prompt("First number?", 1);  
let b = prompt("Second nummber?"2);  
alert(a + b); // 3
```

Comparaison

Quel sera le résultat pour les expressions suivantes :

```
(5 > 4);  
("apple" > "pineapple");  
("2" > "12");  
(undefined == null);  
(undefined === null);  
(null == "\n0\n");  
(null === +"\n0");
```

5 > 4 = true

"apple" > "pineapple" = false

"2" > "12" = true

undefined == null = true

undefined === null = false

null == "\n0\n" = false

null === +"\n0" = false

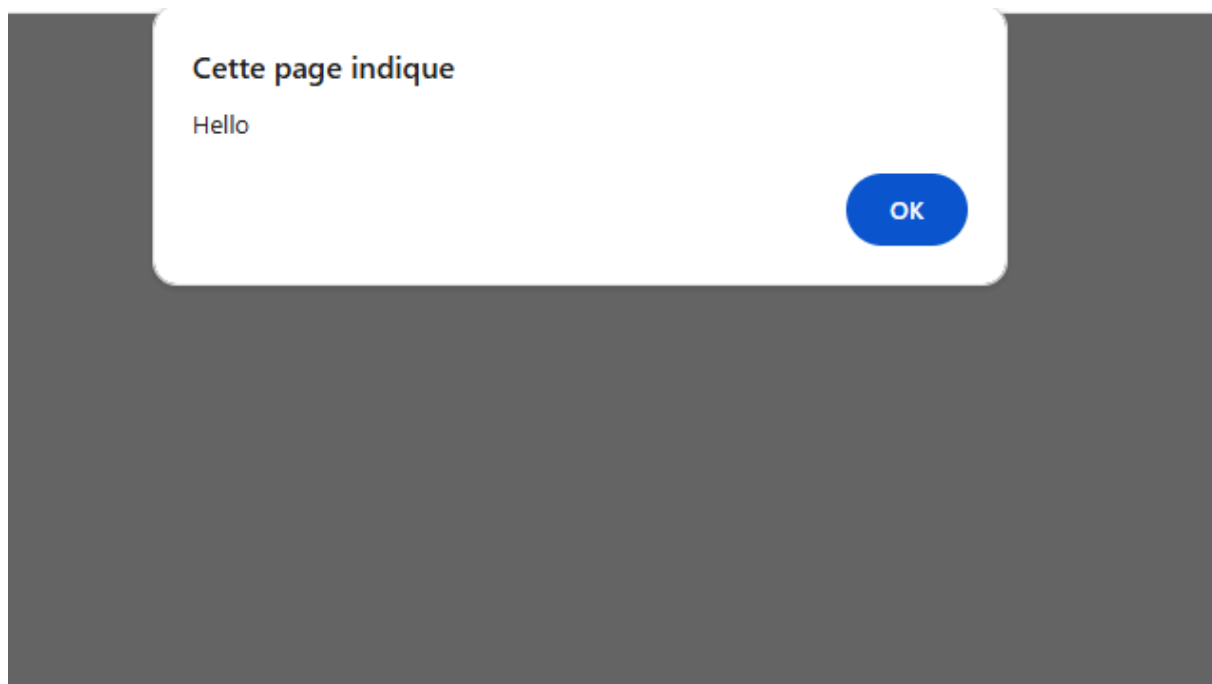
```
true  
false  
true  
true  
false  
false  
false
```

if (une chaîne de caractères avec zéro)

Est-ce que `alert` sera affiché ?

Oui `alert` sera affiché

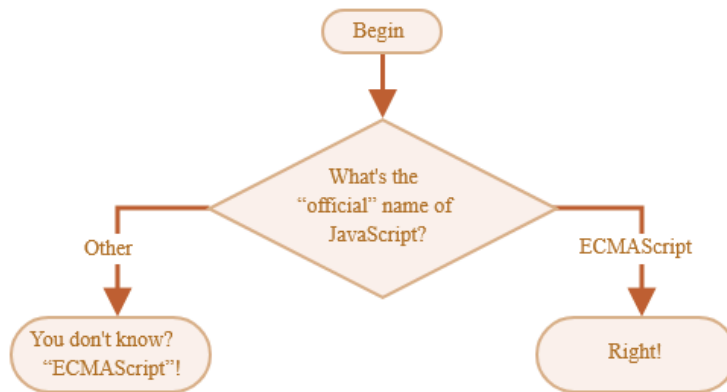
```
if ("0") {  
  console.log('Hello');  
}
```



Le nom de JavaScript

En utilisant la construction `if...else`, écrivez le code qui demande : 'Quel est le nom "officiel" de JavaScript?'

Si le visiteur entre "ECMAScript", alors éditez une sortie "Bonne réponse !", Sinon – retourne "Ne sait pas ? ECMAScript!"



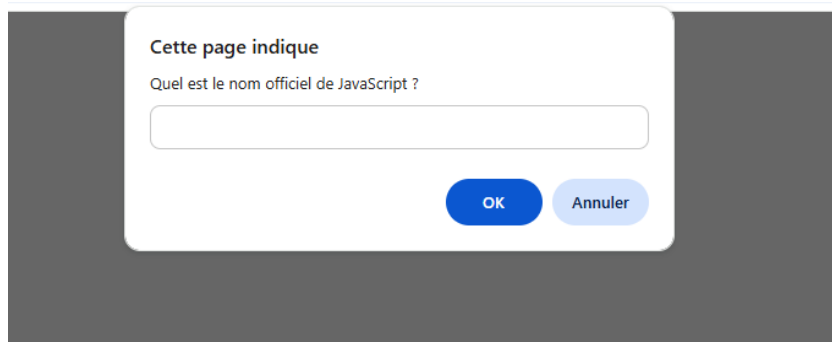
- Le programme demande à l'utilisateur : « Quel est le nom officiel de JavaScript ? » grâce à la fonction prompt.
- Si l'utilisateur répond « ECMAScript » (exactement comme ça), le programme affiche une alerte avec le message « Bonne réponse ! ».
- Sinon, le programme affiche une alerte avec le message « Ne sait pas ? ECMAScript ! ».
- Le programme demande à l'utilisateur d'entrer un nombre.
- Il vérifie ensuite si ce nombre est :
 - supérieur à zéro, alors il affiche une alerte avec 1.
 - inférieur à zéro, alors il affiche une alerte avec -1.
 - égal à zéro, alors il affiche une alerte avec 0.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8" />
  <title>Exercice JavaScript</title>
</head>
<body>
  <script>
    let answer = prompt("Quel est le nom officiel de JavaScript ?");

    if (answer === "ECMAScript") {
      alert("Bonne réponse");
    } else {
      alert("Ne sait pas ? ECMAScript");
    }

    let number = Number(prompt("Entrez un nombre :"));

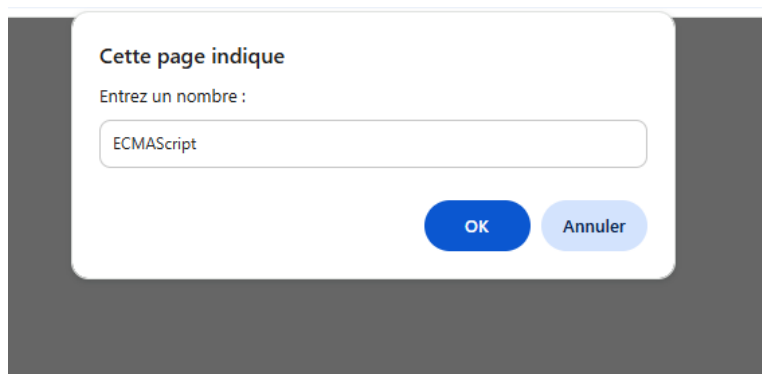
    if (number > 0) {
      alert(1);
    } else if (number < 0) {
      alert(-1);
    } else {
      alert(0);
    }
  </script>
</body>
</html>
```



Cette page indique

Quel est le nom officiel de JavaScript ?

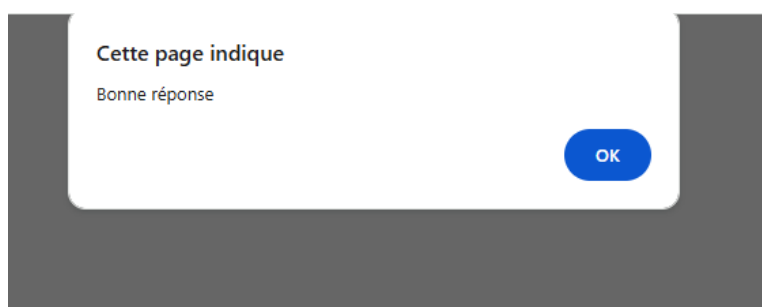
OK Annuler



Cette page indique

Entrez un nombre :

OK Annuler



Cette page indique

Bonne réponse

OK

Cette page indique

Quel est le nom officiel de JavaScript ?

OK Annuler

Cette page indique

Ne sait pas ? ECMAScript

OK

Afficher le signe

Pour cet exercice JavaScript, nous avons créé un petit programme qui demande d'abord à l'utilisateur quel est le nom officiel de JavaScript. Si l'utilisateur répond « ECMAScript », une alerte s'affiche pour indiquer que la réponse est bonne. Sinon, le programme indique que la bonne réponse est « ECMAScript ».

Ensuite, nous demandons à l'utilisateur de saisir un nombre. Le programme vérifie si ce nombre est positif, négatif ou égal à zéro. Selon le résultat, il affiche une alerte avec 1, -1 ou 0.

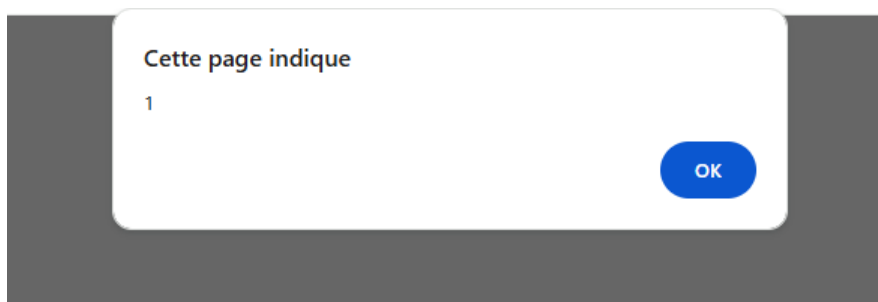
Pour réaliser cela, nous avons utilisé `prompt` pour récupérer les saisies de l'utilisateur, `if..else if..else` pour tester les conditions, et `alert` pour afficher le résultat.

Nous avons testé le programme dans un navigateur, car `alert` et `prompt` ne fonctionnent pas dans Node.js.

Le programme fonctionne correctement et permet de comprendre comment JavaScript gère les conditions et les entrées utilisateur.

```
let number = Number(prompt("Entrez un nombre :"));

if (number > 0) {
  alert(1);
} else if (number < 0) {
  alert(-1);
} else {
  alert(0);
}
```



Réécrire 'if' en '?'

Réécrire ce `if..else` en utilisant plusieurs opérateurs ternaires `'?'`.

Pour plus de lisibilité, il est recommandé de diviser le code en plusieurs lignes.

```
let message =  
  (login == 'Employee') ? 'Hello' :  
  (login == 'Director') ? 'Greetings' :  
  (login == '') ? 'No login' :  
  '';
```

Quel est le résultat de OR ?

Qu'est-ce que le code ci-dessous va sortir ?

```
1 alert( null || 2 || undefined );
```

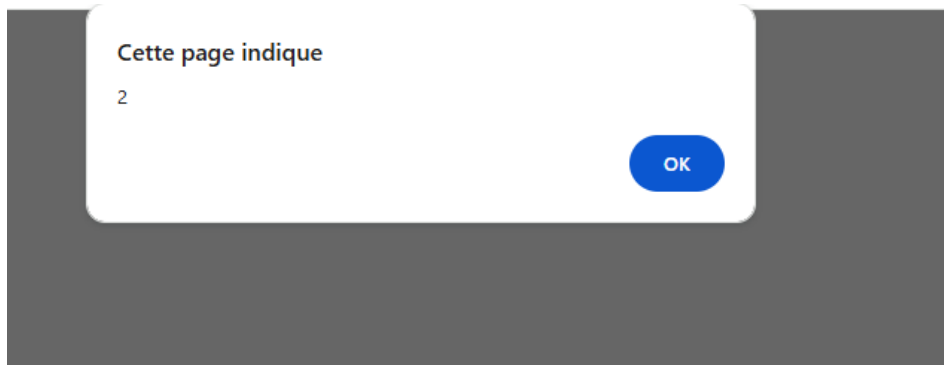
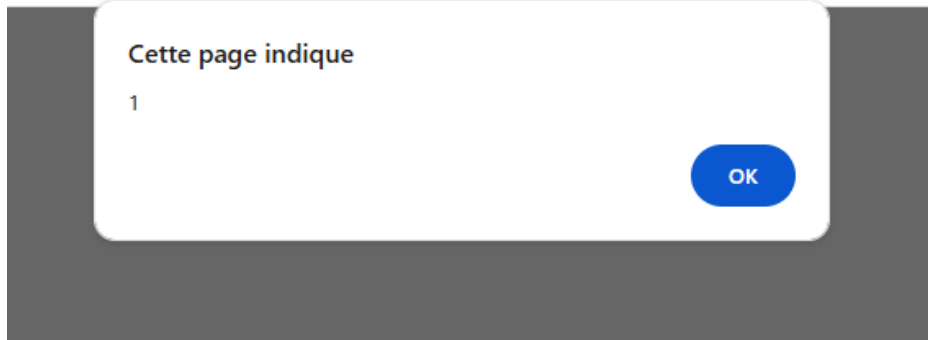
L'opérateur || renvoie la première valeur "vraie" qu'il rencontre. Dans ce cas :

- null est une valeur fausse, donc il passe à la suivante.
- 2 est une valeur vraie, donc l'opérateur s'arrête là et renvoie 2.
- undefined n'est pas évalué, car une valeur vraie a déjà été trouvée.

Ainsi, le résultat affiché par l'alerte est 2.

Quel est le résultat des alertes OR ?

Qu'est-ce que le code ci-dessous va sortir ?



Premièrement, `alert(1)` s'exécute et affiche 1, mais retourne `undefined`.
Ensuite, l'opérateur `||` passe à la valeur suivante, 2, qui est vraie.
Donc la deuxième alerte affiche 2.

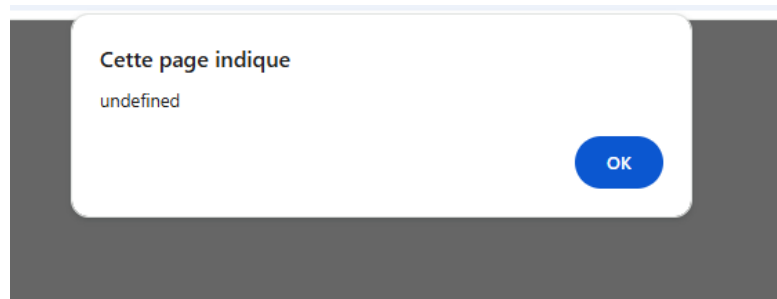
`alert(3)` n'est jamais exécuté, car le `||` s'arrête dès qu'il trouve une valeur vraie.

Ainsi, nous comprenons que `||` renvoie toujours la première valeur vraie et que certaines expressions peuvent ne pas être évaluées si une valeur vraie est déjà trouvée.

Quel est le résultat des alertes AND ?

Qu'est-ce que ce code va afficher ?

```
</head>
<body>
  <script>
    alert( alert(1) && alert(2) );
  </script>
</body>
</html>
```



Nous voyons d'abord que `alert(1)` va s'exécuter.
Cela affiche 1 à l'écran.
Mais après avoir affiché, `alert(1)` retourne `undefined`.

Ensuite, l'opérateur `&&` regarde la valeur à gauche.
Comme `undefined` est considéré comme faux, il n'exécute pas `alert(2)`.

Le résultat de OR AND OR

Quel sera de résultat ?

Le résultat de :

```
alert( null || 2 && 3 || 4 );
```

sera 3.

```
<script>
  alert( null || 2 && 3 || 4 );
</script>
```

Cette page indique

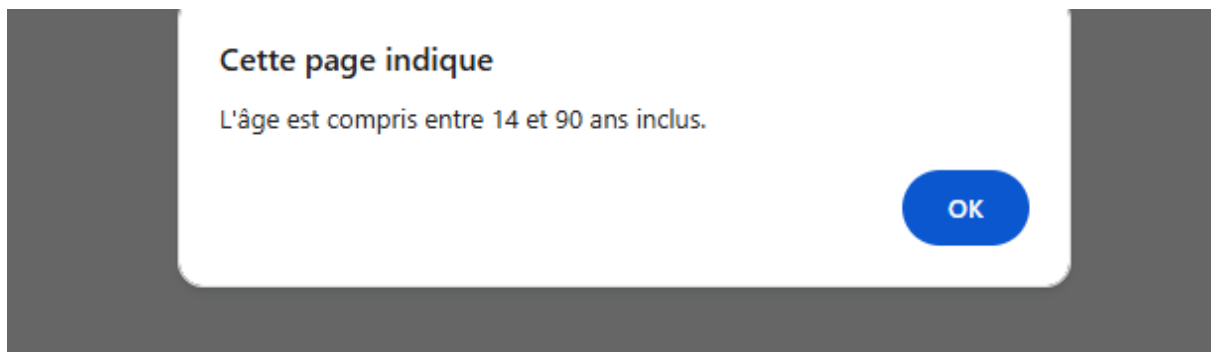
3

OK

Vérifiez la plage entre

Ecrivez une condition "if" pour vérifier que l'âge est compris entre 14 et 90 ans inclus.

"Inclus" signifie que l'âge peut atteindre les 14 ou 90 ans.



nous voyons que le navigateur affiche une alerte avec le message « L'âge est compris entre 14 et 90 ans inclus. ».

- Cela s'explique parce que la variable age vaut 25, ce qui est supérieur ou égal à 14 et inférieur ou égal à 90.
- L'opérateur && vérifie que les deux conditions sont vraies en même temps.
- Comme c'est le cas, la première branche du if s'exécute.

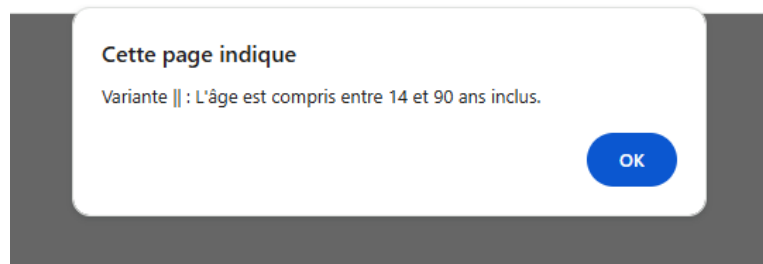
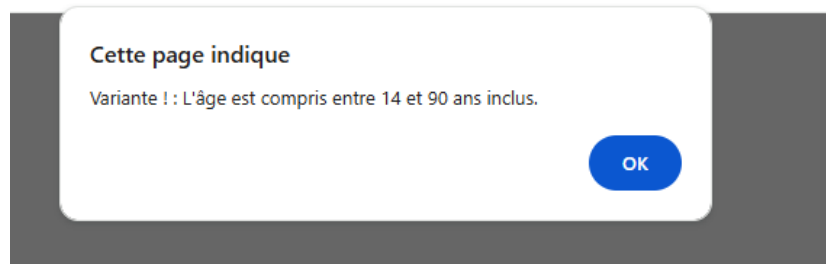
Si nous changions la valeur de age à une valeur inférieure à 14 ou supérieure à 90, l'alerte afficherait alors « L'âge est en dehors de la plage autorisée.

Vérifiez à l'extérieur de la page

Nous avons écrit un code qui vérifie si notre âge est compris entre 14 et 90 ans.

1. Nous avons créé une variable age qui contient notre âge, par exemple 25.
2. Nous avons utilisé deux conditions if :
 - La première utilise ! pour dire « si l'âge n'est pas entre 14 et 90 ».
 - La seconde utilise || pour vérifier directement si l'âge est moins que 14 ou plus que 90.

```
if (!(age >= 14 && age <= 90)) {  
    alert("Variante ! : L'âge n'est pas compris entre 14 et 90 ans inclus.");  
} else {  
    alert("Variante ! : L'âge est compris entre 14 et 90 ans inclus.");  
}  
  
if (age < 14 || age > 90) {  
    alert("Variante || : L'âge n'est pas compris entre 14 et 90 ans inclus.");  
} else {  
    alert("Variante || : L'âge est compris entre 14 et 90 ans inclus.");  
}
```



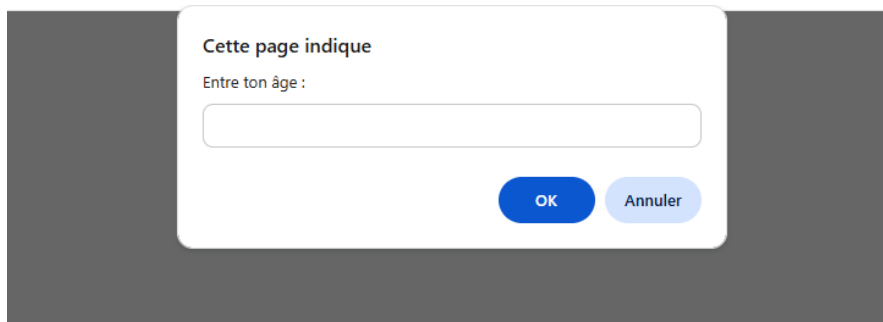
Une question à propos de “if”

Lesquelles de ces alertes vont s'exécuter ?

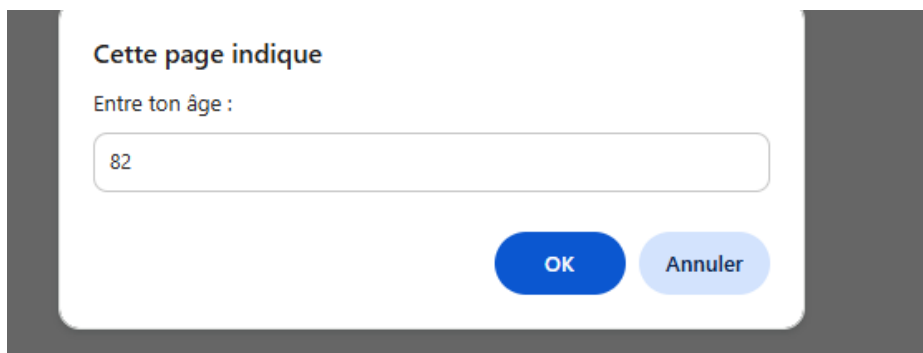
Les alertes qui vont s'exécuter sont : 'first' et 'third'.

Quels seront les résultats des expressions à l'intérieur de if (...) ?

Les résultats sont -1, 0, et 1. Seules les expressions donnant -1 et 1 déclenchent l'alerte.



A screenshot of a JavaScript alert dialog box. The title bar says "Cette page indique". The main text says "Entre ton âge :". Below the text is an empty text input field. At the bottom right are two buttons: "OK" (blue) and "Annuler" (light blue).



A screenshot of a JavaScript alert dialog box, similar to the one above. The title bar says "Cette page indique". The main text says "Entre ton âge :". Below the text is a text input field containing the number "82". At the bottom right are two buttons: "OK" (blue) and "Annuler" (light blue).

Check the login

Nous allons conformément à la consigne écrire le code correspondant :

De sorte, nous souhaitons que la page s'exécute comme la manière suivante :

Si l'utilisateur clique sur Annuler ou entre une chaîne vide : "Canceled"

Si le nom est "Admin", on demande le mot de passe

Si le mot de passe est "TheMaster" : "Welcome!"

Si le mot de passe est faux : "Wrong password"

Si le nom n'est pas "Admin" : "I don't know you"

```
<script>
let userName = prompt("Who's there?");

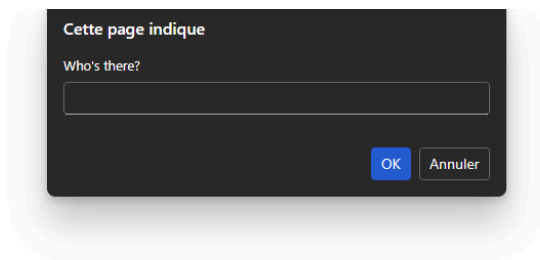
if (userName === "Admin") {

    let password = prompt("Password?");

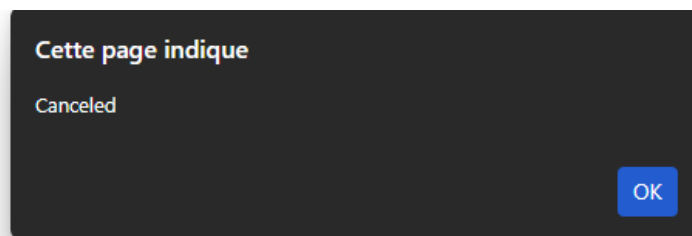
    if (password === "TheMaster") {
        alert("Welcome!");
    } else if (password === "" || password === null) {
        alert("Canceled");
    } else {
        alert("Wrong password");
    }

} else if (userName === "" || userName === null) {
    alert("Canceled");
} else {
    alert("I don't know you");
}
</script>
```

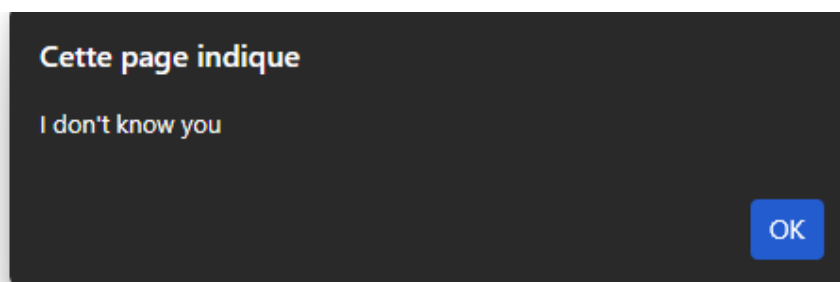
Debut



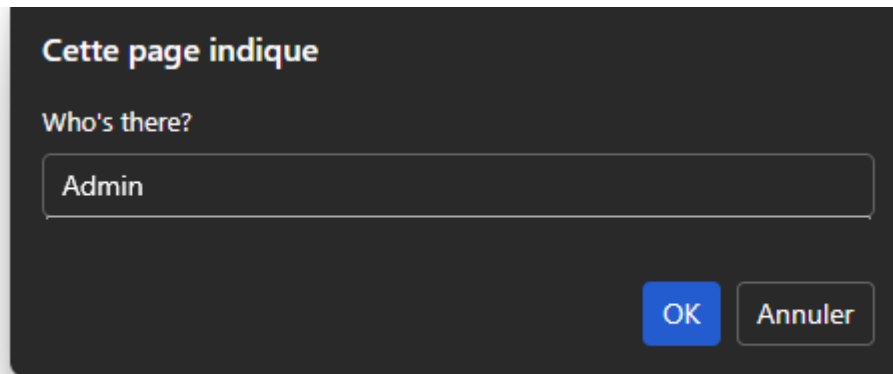
En cliquant sur annuler cela affiche :



Si nous rentrons "admi" à la place de "Admin"



Bon identifiant



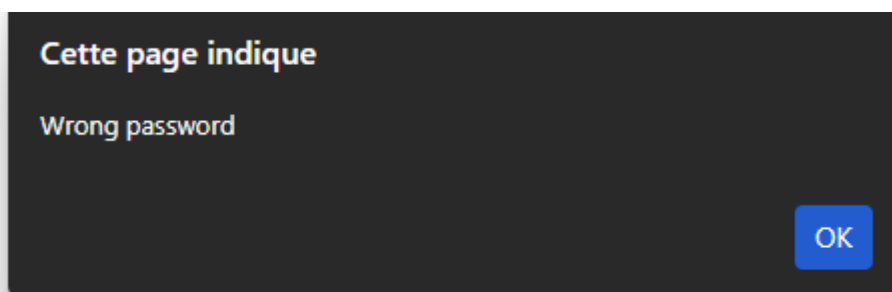
A dark-themed modal dialog box with the title "Cette page indique" in bold. Below the title is the label "Who's there?". A text input field contains the word "Admin". At the bottom right, there are two buttons: a blue "OK" button and a grey "Annuler" button.

La page nous demande ensuite le mot de passe



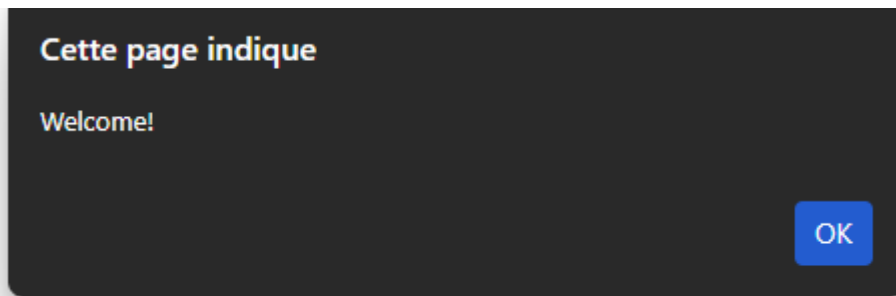
A dark-themed modal dialog box with the title "Cette page indique" in bold. Below the title is the label "Password?". There is an empty text input field. At the bottom right, there are two buttons: a blue "OK" button and a grey "Annuler" button.

Mauvais mot de passe



A dark-themed modal dialog box with the title "Cette page indique" in bold. Below the title is the text "Wrong password". At the bottom right, there is a single blue "OK" button.

Mot de passe "TheMaster"



Fondementaux 3

Dernière valeur de boucle

Quelle est la dernière valeur affichée par ce code ? Pourquoi ?

```
1 let i = 3;  
2  
3 while (i) {  
4   alert( i-- );  
5 }
```

La dernière valeur affichée est 1.

Au départ, la variable `i` vaut 3. La boucle `while (i)` s'exécute tant que `i` est différent de 0. À chaque tour, le programme affiche la valeur de `i`, puis la diminue de 1 grâce à `i--`. Ainsi, les valeurs affichées sont 3, puis 2, puis 1. Quand `i` devient 0, la condition du `while` n'est plus vraie et la boucle s'arrête. C'est pour cela que la dernière valeur affichée est 1..

Quelles valeurs affiche la boucle while ?

A votre avis, quelles sont les valeurs affichées pour chaque boucle ? Notez-les puis comparez avec la réponse.

Les deux boucles affichent-elles les mêmes valeurs dans l'`alert` ou pas ?

1.

Le préfixe sous forme `++i` :

```
1 let i = 0;
2 while (++i < 5) alert( i );
```

2.

Le postfixe sous forme `i++` :

```
1 let i = 0;
2 while (i++ < 5) alert( i );
```

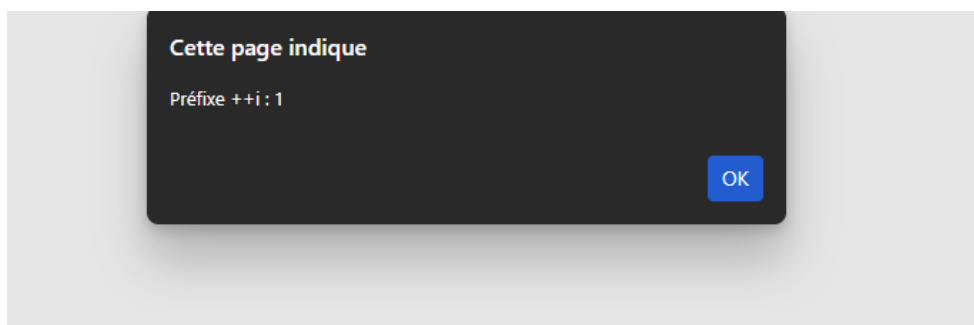
Nous avons testé deux variantes avec `i = 0`. La première utilise le préfixe `++i < 5` : elle incrémente `i` avant la comparaison, affichant ainsi **1, 2, 3 et 4**. La seconde utilise le postfixe `i++ < 5` : elle compare la valeur avant de l'augmenter, ce qui permet d'afficher **1, 2, 3, 4 et 5**. En conclusion, les deux boucles n'affichent pas les mêmes valeurs car le postfixe permet une itération de plus.

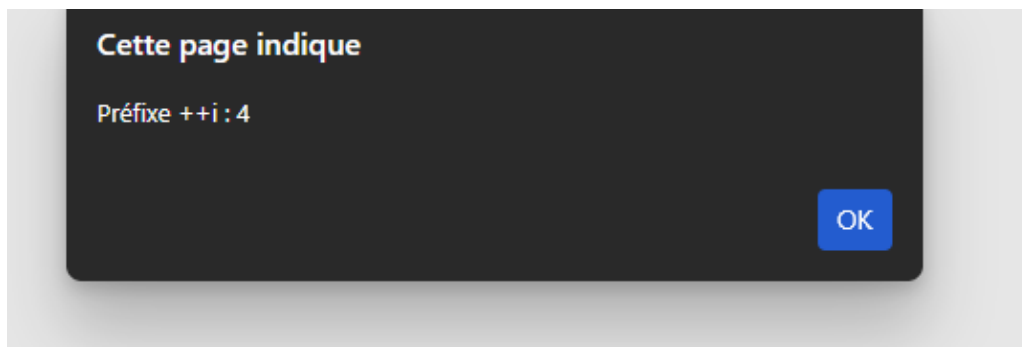
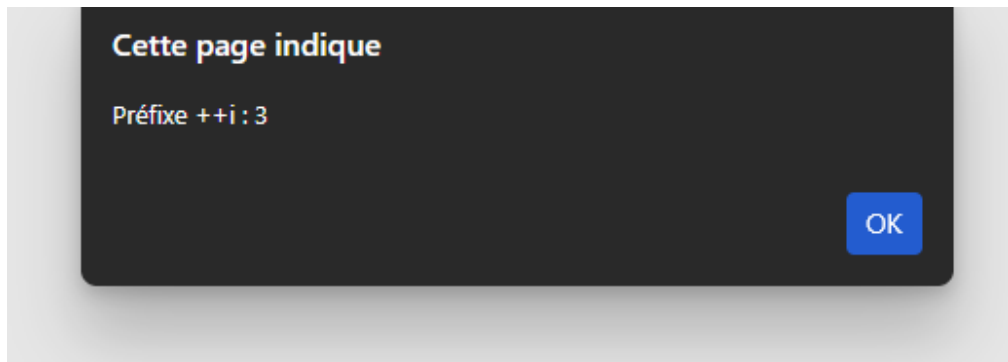
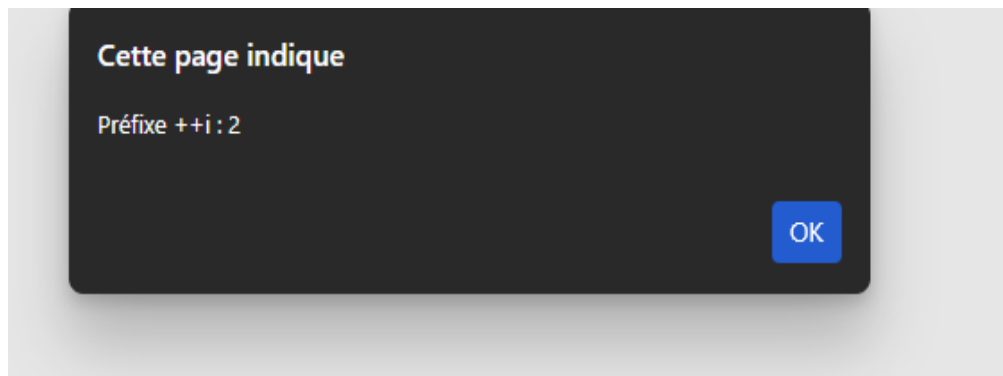
La variable `i` commence à 0.

Avec `++i`, `i` est **incrémenté avant** d'être comparé à 5.

La boucle continue et affichera ensuite **2, 3, 4**, puis s'arrête quand `i` devient 5.

la boucle affiche les valeurs **1 à 4**, car l'incrémentation se fait avant le test.





Nous avons testé la boucle `while` avec `i++`

La variable `i` commence à 0.

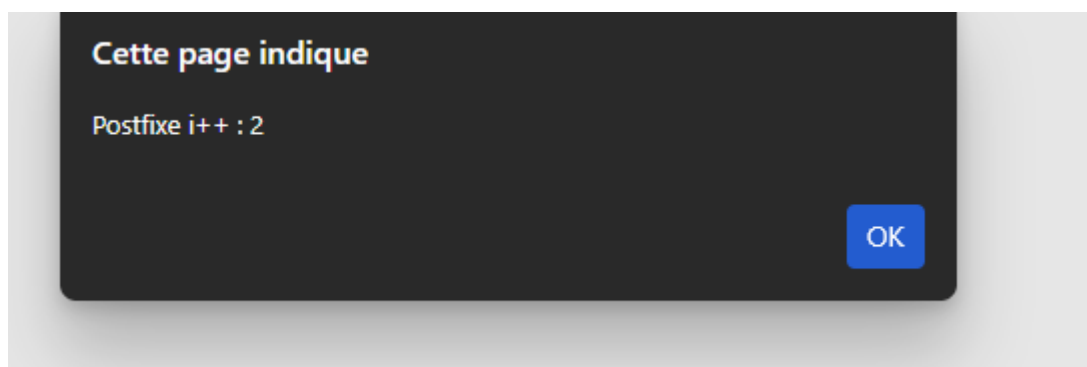
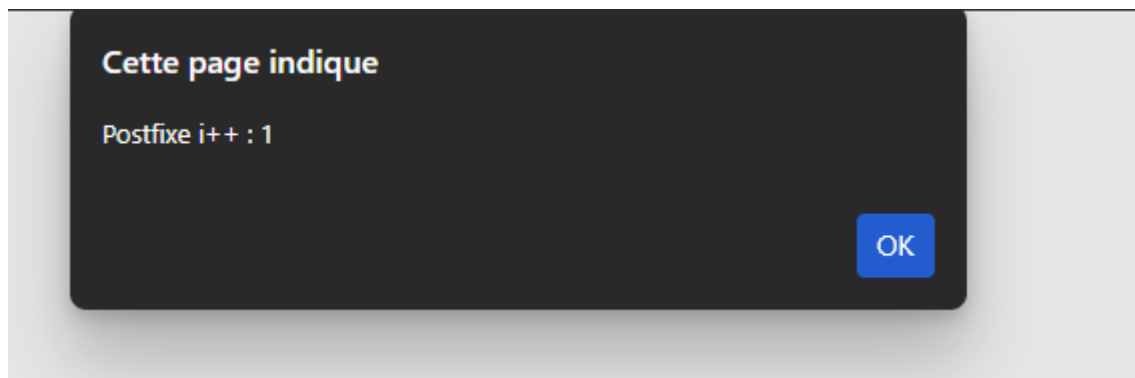
Avec `i++`, `i` est comparé à 5 avant d'être incrémenté.

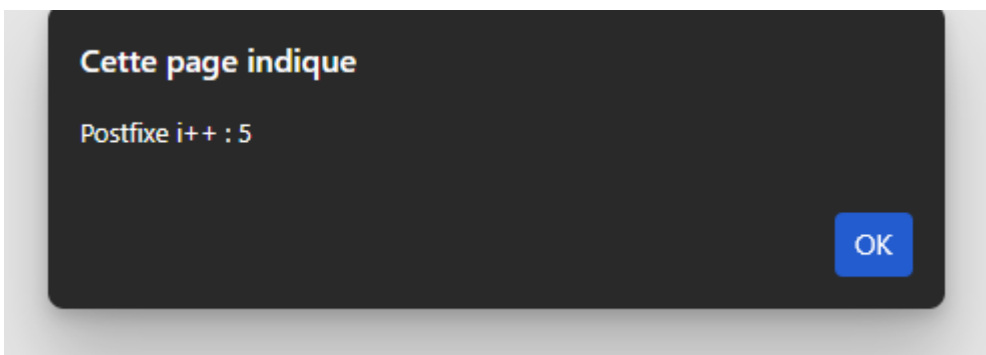
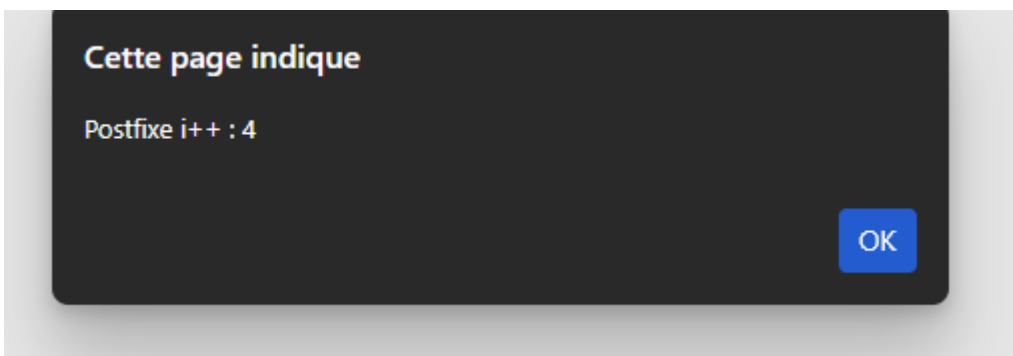
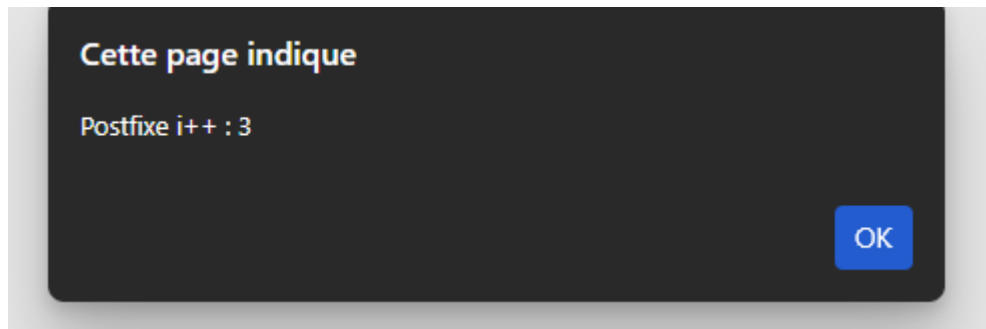
La première alerte affiche donc 1.

En cliquant sur 'ok', la boucle affiche ensuite 2, 3, 4 et 5.

La boucle s'arrête quand **i devient 6**, car la condition `i < 5` **n'est plus vraie**.

la boucle affiche les valeurs 1 à 5, car l'incrémentation se fait après le test.





Quelles valeurs sont affichées par la boucle "for" ?

Quelles valeurs sont affichées par la boucle "for" ?

Pour chaque boucle, notez les valeurs qui vont s'afficher. Ensuite, comparez avec la réponse.

Les deux boucles `alert` les mêmes valeurs ou pas ?

1.

La forme postfix :

```
1 for (let i = 0; i < 5; i++) alert( i );
```

2.

La forme préfix :

```
1 for (let i = 0; i < 5; ++i) alert( i );
```

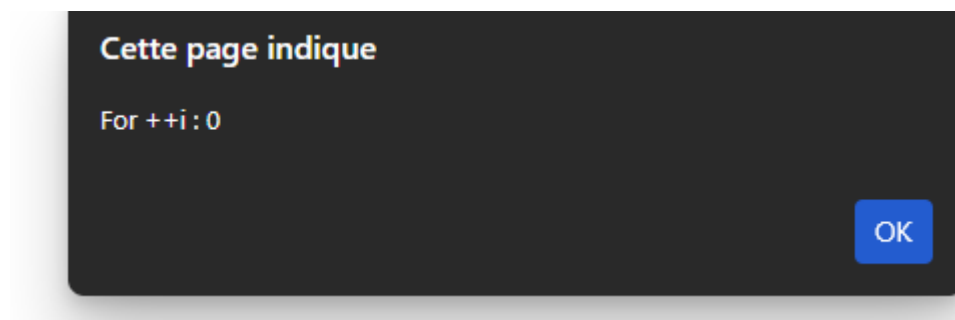
—

Cette page indique

For i++ : 0

OK

Nous avons testé la boucle for avec i++. La variable i commence à 0 et la boucle s'exécute tant que i < 5. La première alerte affiche 0. À chaque clic sur OK, la boucle continue et affichera ensuite 1, 2, 3 et 4. La boucle s'arrête quand i devient 5. Conclusion : la boucle affiche les valeurs 0 à 4 et l'incrémentation se fait après l'affichage à cause du i++.

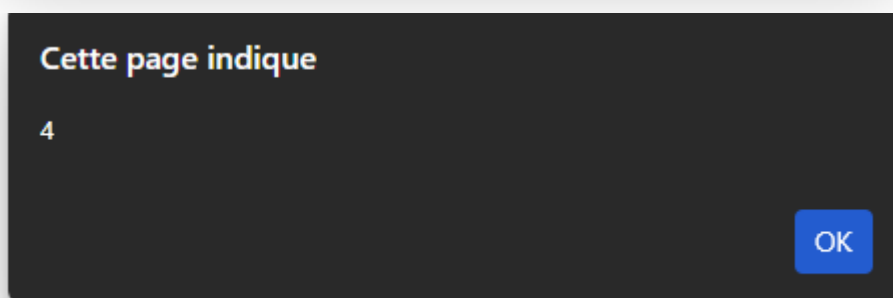
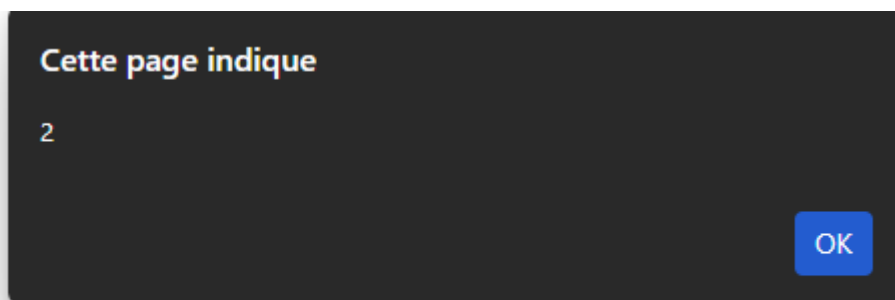


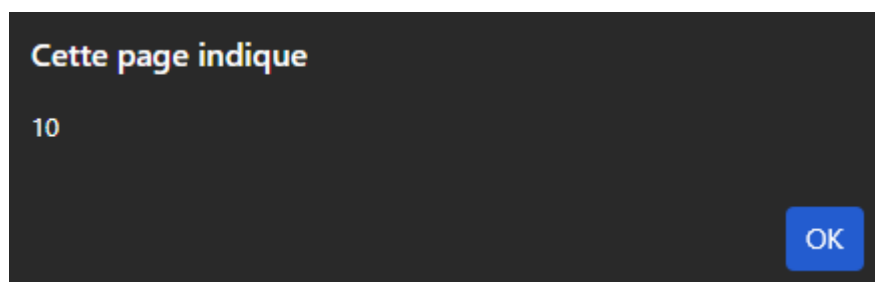
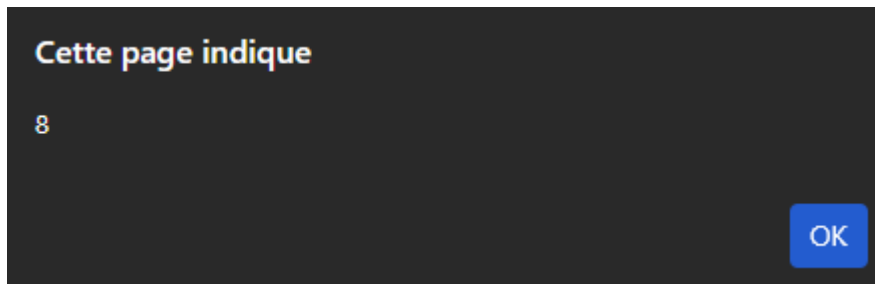
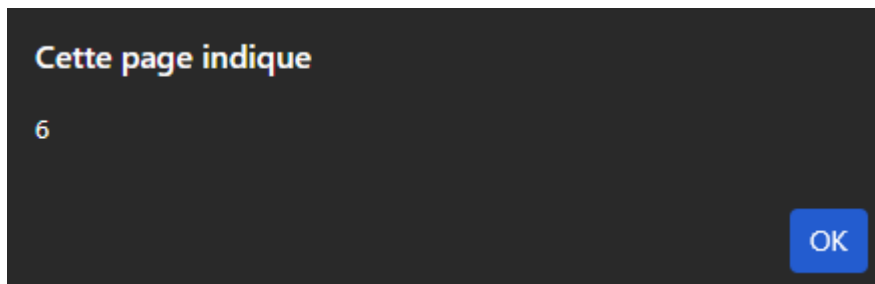
Nous avons ensuite testé la boucle `for` avec `++i`. La variable `i` commence à 0 et la boucle s'exécute tant que `i < 5`. La première alerte affiche `0`. À chaque clic sur OK, la boucle continue et affichera ensuite 1, 2, 3 et 4. La boucle s'arrête quand `i` devient 5. **Conclusion** : la boucle affiche les valeurs 0 à 4 et l'incrémement se fait avant le test suivant à cause du `++i`.

Extraire les nombres pairs dans la boucle

Extraire les nombres pairs dans la boucle

Utilisez la boucle `for` pour afficher les nombres pairs de 2 à 10.





Nous avons testé la boucle for pour afficher les nombres pairs de 2 à 10. La première valeur affichée est 2, car la variable `i` commence à 2. La boucle s'exécute tant que `i <= 10` et à chaque clic sur 'ok', `i` augmente de 2. Les valeurs suivantes affichées seront donc 4, 6, 8 et 10.

La boucle affiche tous les nombres pairs de 2 à 10, une valeur à la fois avec `alert()`.

Remplacer “for” par “while”

Remplacer "for" par "while"

Réécrivez le code en modifiant la boucle `for` en `while` sans modifier son comportement (la sortie doit rester la même).

```
1 for (let i = 0; i < 3; i++) {  
2   alert( `number ${i}!` );  
3 }
```

la variable `i` commence à 0 et la condition `i < 3` est fausse dès le départ, donc la boucle ne s'exécute jamais. Le comportement de la boucle `while` est exactement le même que celui du `for`.

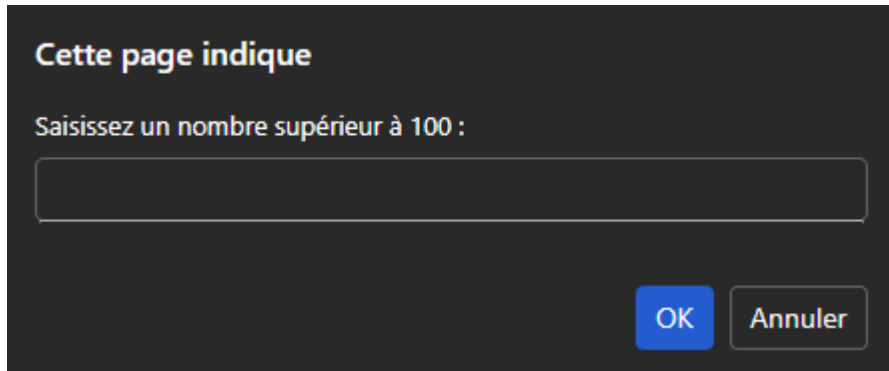
Répéter jusqu'à ce que l'entrée soit correcte

Répéter jusqu'à ce que l'entrée soit correcte

Ecrivez une boucle qui demande un nombre supérieur à 100. Si le visiteur saisit un autre numéro, demandez-lui de le saisir à nouveau.

La boucle doit demander un numéro jusqu'à ce que le visiteur saisisse un nombre supérieur à 100 ou annule l'entrée/entre une ligne vide.

Ici, nous pouvons supposer que le visiteur ne saisit que des chiffres. Il n'est pas nécessaire de mettre en œuvre un traitement spécial pour une entrée non numérique dans cette tâche.



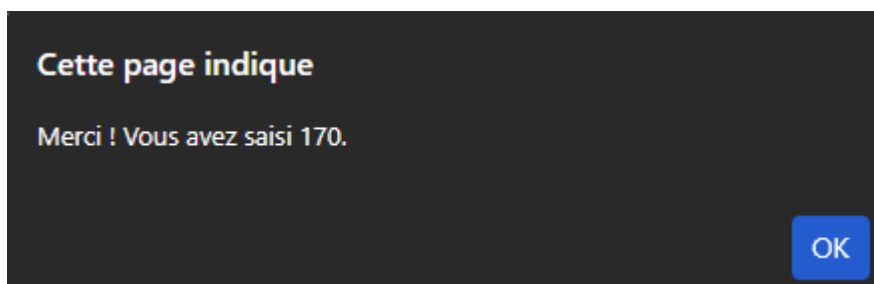
Cette page indique

Saisissez un nombre supérieur à 100 :

OK Annuler

Nous avons écrit une boucle qui demande à l'utilisateur de saisir un nombre supérieur à 100. La boucle continue tant que le nombre est inférieur ou égal à 160. Si l'utilisateur clique sur Annuler ou laisse la saisie vide, la boucle s'arrête. Dès qu'un nombre supérieur à 160 est saisi, la boucle se termine et affiche un message de remerciement.

Exemple en saisissant 170



Cette page indique

Merci ! Vous avez saisi 170.

OK

Extraire des nombres premiers

Extraire des nombres premiers

Un nombre entier supérieur à 1 est appelé un **Nombre premier** s'il ne peut être divisé sans reste par rien d'autre que 1 et lui-même.

En d'autres termes, $n > 1$ est un nombre premier s'il ne peut être divisé de manière égale par autre chose que 1 et n .

Par exemple, 5 est un nombre premier, car il ne peut pas être divisé sans reste par 2, 3 et 4.

Écrivez un code qui produit les nombres premiers dans l'intervalle 2 à n .

Pour $n = 10$, le résultat sera 2, 3, 5, 7.

P.S. Le code devrait fonctionner pour n'importe quel n et aucune valeur fixe ne doit être codé en dur.

Cette page indique

Entrez un nombre supérieur à 2 pour afficher les nombres premiers :

Le programme demande à l'utilisateur un nombre supérieur ou égal à 2, puis il vérifie chaque nombre de 2 jusqu'à ce nombre. Pour chaque nombre, il teste s'il est divisible par un autre nombre que 1 et lui-même en utilisant une boucle jusqu'à la racine carrée du nombre. Si le nombre n'a aucun diviseur autre que 1 et lui-même, il est ajouté à la liste des nombres premiers. À la fin, le programme affiche tous les nombres premiers trouvés.

Exemple avec 4 :

Cette page indique

Nombres premiers jusqu'à 4 : 2, 3

Écrivez le code en utilisant `if..else` qui correspondrait au `switch` suivant :

```
1  switch (browser) {  
2    case 'Edge':  
3      alert( "You've got the Edge!" );  
4      break;  
5  
6    case 'Chrome':  
7    case 'Firefox':  
8    case 'Safari':  
9    case 'Opera':  
10     alert( 'Okay we support these browsers too' );  
11     break;  
12  
13   default:  
14     alert( 'We hope that this page looks ok!' );  
15 }
```

Cette page indique

Quel navigateur utilisez-vous ?

OK Annuler

Nous avons réécrit le switch en utilisant if..else. Si le navigateur est "Edge", un message spécifique s'affiche. Si le navigateur est "Chrome", "Firefox", "Safari" ou "Opera", un autre message s'affiche. Pour tous les autres navigateurs, le message par défaut s'affiche. Cela permet de gérer tous les cas de manière équivalente au switch.

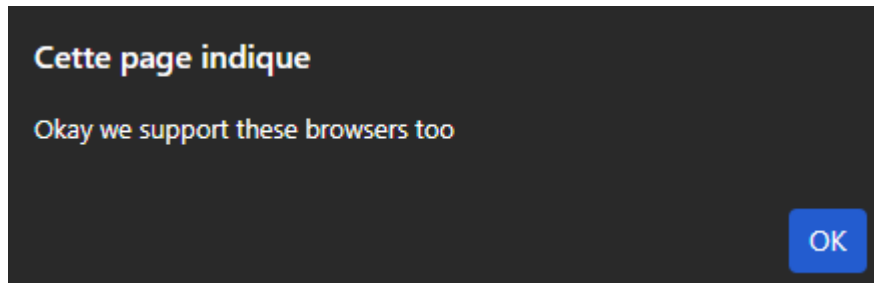
Exemple avec Edge

Cette page indique

You've got the Edge!

OK

Exemple avec Firefox



Réécrire le "if" dans un "switch"

Réécrire le "if" dans un "switch"

Réécrivez le code ci-dessous en utilisant une seule instruction `switch` :

```
1 let a = +prompt('a?', '');
2
3 if (a == 0) {
4   alert( 0 );
5 }
6 if (a == 1) {
7   alert( 1 );
8 }
9
10 if (a == 2 || a == 3) {
11   alert( '2,3' );
12 }
```

Cette page indique

a?

OK Annuler

Nous avons réécrit les conditions if en une seule instruction switch. Si a vaut 0, le message 0 s'affiche, si a vaut 1, le message 1 s'affiche, et si a vaut 2 ou 3, le message '2,3' s'affiche. Pour toutes les autres valeurs, un message par défaut est affiché.

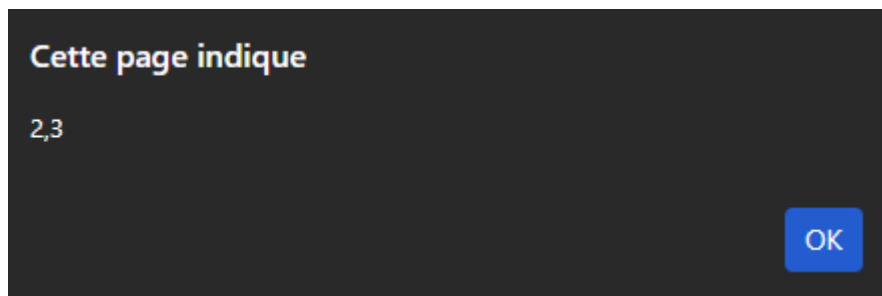
Exemple avec 0

Cette page indique

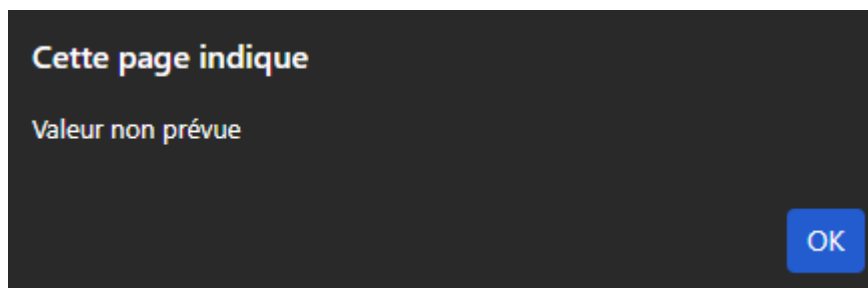
0

OK

Exemple avec 2



Exemple avec 8



Est-ce que “else” est requis ?

La fonction suivante renvoie `true` si le paramètre `age` est supérieur à `18`.

Sinon, il demande une confirmation et renvoie son résultat :

```
1 function checkAge(age) {  
2   if (age > 18) {  
3     return true;  
4   } else {  
5     // ...  
6     return confirm('Did parents allow you?');  
7   }  
8 }
```

La fonction fonctionnera-t-elle différemment si `else` est supprimé ?

Cette page indique

Quel est votre âge ?

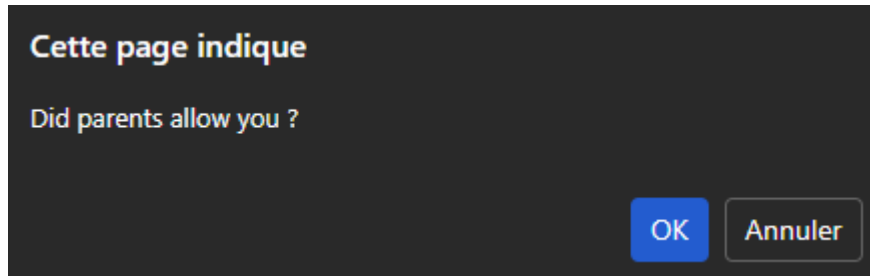
Cette page indique

Accès autorisé

Nous avons testé la fonction “checkAge” qui renvoie `true` si l’âge est supérieur à 18 et demande une confirmation sinon. L’instruction `else` n’est pas nécessaire car le `return` arrête la fonction quand la condition est vraie. Si la condition n’est pas remplie, le code suivant s’exécute automatiquement. La fonction fonctionne donc de la même manière avec ou sans `else`.

En cas d'âge égal ou inférieur à 18

La fonction demande une confirmation via confirm.



La fonction fonctionnera-t-elle différemment si `else` est supprimé ?

```
1 function checkAge(age) {  
2   if (age > 18) {  
3     return true;  
4   }  
5   // ...  
6   return confirm('Did parents allow you?');  
7 }
```

Existe-t-il une différence dans le comportement de ces deux variantes ?

Nous avons testé la fonction `checkAge` sans utiliser `else`. Si l'âge est supérieur à 18, la fonction renvoie directement `true` et affiche "Accès autorisé". Sinon, elle demande une confirmation via une boîte de dialogue et affiche "Accès autorisé" ou "Accès refusé" selon la réponse. Supprimer `else` ne change pas le comportement car le `return` dans le `if` arrête la fonction.

La différence dans le code seulement syntaxique : avec `else` c'est explicite, sans `else` c'est plus court, mais la fonction se comporte de la même manière.

Réécrivez la fonction en utilisant '?' ou '||'

Réécrivez la fonction en utilisant '?' ou '||'

La fonction suivante renvoie `true` si le paramètre `age` est supérieur à `18`.

Sinon, il demande une confirmation et renvoie le résultat.

```
1 function checkAge(age) {  
2   if (age > 18) {  
3     return true;  
4   } else {  
5     return confirm('Did parents allow you?');  
6   }  
7 }
```

Réécrivez-le, pour effectuer la même chose, mais sans `if`, et en une seule ligne.

Faites deux variantes de `checkAge` :

1. En utilisant un opérateur point d'interrogation `?`
2. En utilisant OU `||`

Nous avons réécrit la fonction `checkAge` en une seule ligne sans utiliser `if`. La première variante utilise l'opérateur conditionnel `?` pour renvoyer `true` si l'âge est supérieur à 18, sinon demander confirmation.

```
<script>  
function checkAge(age) {  
  return (age > 18) ? true : confirm("Did parents allow you?");  
}  
  
let age = +prompt("Quel est votre âge ?", "");  
alert(checkAge(age) ? "Accès autorisé" : "Accès refusé");  
</script>
```

Exemple avec 30

Cette page indique

Accès autorisé

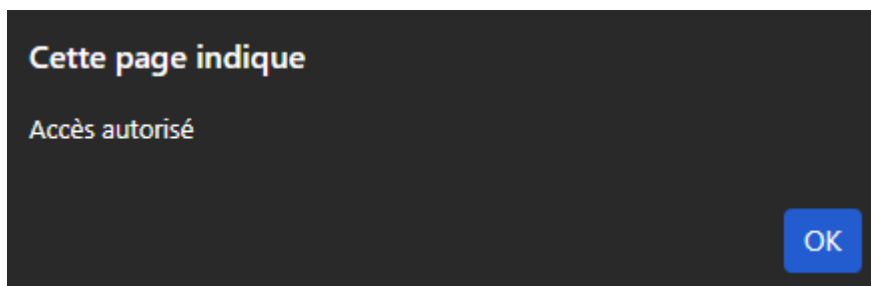
OK

La deuxième variante utilise `||` : si `age > 18` est vrai, la fonction renvoie `true`, sinon elle demande confirmation. Les deux variantes font exactement la même chose.

```
<script>
function checkAge(age) {
  return (age > 18) || confirm("Did parents allow you?");
}

let age = +prompt("Quel est votre âge ?", "");
alert(checkAge(age) ? "Accès autorisé" : "Accès refusé");
</script>
```

Exemple avec 30



Fonction min(a,b)

Ecrivez une fonction `min(a, b)` qui renvoie le plus petit des deux nombres `a` et `b`.

Par exemple :

```
1 min(2, 5) == 2
2 min(3, -1) == -1
3 min(1, 1) == 1
```

```
<script>

function min(a, b) {
    return (a < b) ? a : b;
}

alert(`min(2, 5) = ${min(2, 5)}`); // Affiche 2
alert(`min(3, -1) = ${min(3, -1)}`); // Affiche -1
alert(`min(1, 1) = ${min(1, 1)}`); // Affiche 1
</script>
```

La fonction `min(a, b)` compare deux nombres et renvoie le plus petit. Par exemple, `min(2, 5)` renvoie 2, `min(3, -1)` renvoie -1 et `min(1, 1)` renvoie 1. La fonction fonctionne correctement pour tous les cas.

Cette page indique

`min(2, 5) = 2`

OK

Cette page indique

`min(3, -1) = -1`

OK

Cette page indique

`min(1, 1) = 1`

OK

Fonction `pow(x,n)`

Fonction `pow(x,n)`

Ecrivez une fonction `pow(x, n)` qui renvoie `x` à la puissance `n`. Ou, autrement dit, multiplie `x` par lui-même `n` fois et renvoie le résultat.

```
1 pow(3, 2) = 3 * 3 = 9
2 pow(3, 3) = 3 * 3 * 3 = 27
3 pow(1, 100) = 1 * 1 * ... * 1 = 1
```

Créez une page Web qui demande (prompt) `x` et `n`, puis affiche le résultat de `pow(x, n)`.

P.S. Dans cette tâche, la fonction ne doit prendre en charge que les valeurs naturelles de `n` : entiers supérieurs à 1.

```
<script>

function pow(x, n) {
  let result = 1;
  for (let i = 0; i < n; i++) {
    result *= x;
  }
  return result;
}

let x = +prompt("Entrez la base x :");
let n = +prompt("Entrez l'exposant n (entier naturel ≥ 0) :");

if (n < 0 || !Number.isInteger(n)) {
  alert("L'exposant doit être un entier naturel supérieur ou égal à 0.");
} else {
  alert(`${x}^${n} = ${pow(x, n)}`);
}

</script>
```

Cette page indique

Entrez la base x :

La fonction $\text{pow}(x, n)$ calcule x multiplié par lui-même n fois et renvoie le résultat. La page demande à l'utilisateur de saisir x et n et affiche le résultat. Elle fonctionne pour tous les exposants naturels ($n \geq 0$).

Cette page indique

Entrez l'exposant n (entier naturel ≥ 0) :

Cette page indique

$3^2 = 9$

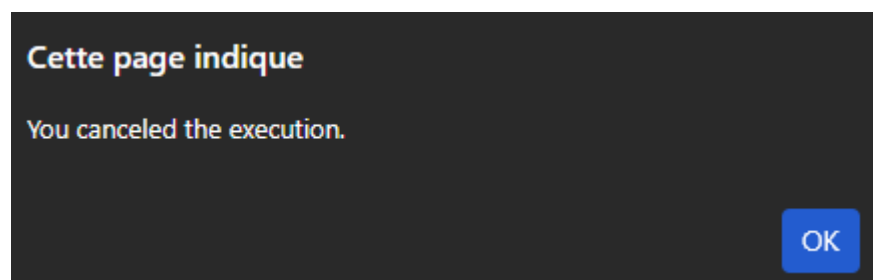
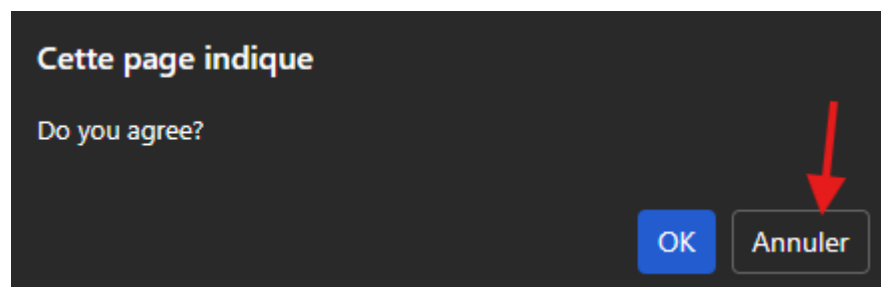
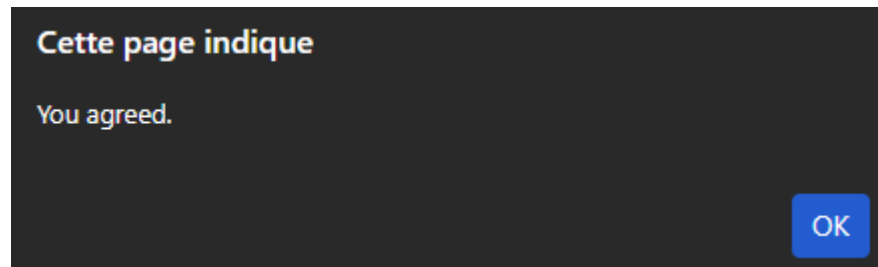
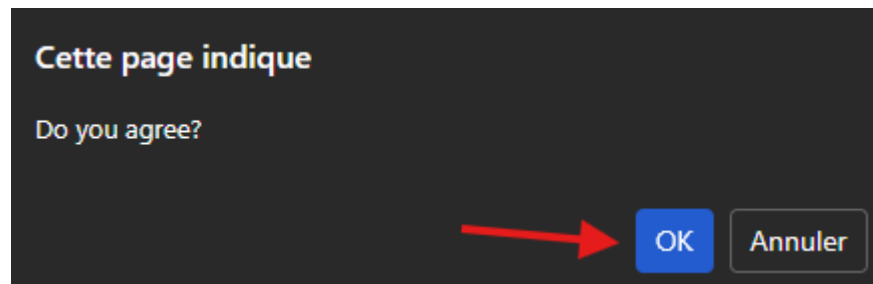
Réécrire avec les fonctions fléchées

Remplacez les expressions de fonction par des fonctions fléchées dans le code ci-dessous :

```
1 function ask(question, yes, no) {  
2   if (confirm(question)) yes();  
3   else no();  
4 }  
5  
6 ask(  
7   "Do you agree?",  
8   function() { alert("You agreed."); },  
9   function() { alert("You canceled the execution."); }  
10 );
```

```
<script>  
  
function ask(question, yes, no) {  
  if (confirm(question)) yes();  
  else no();  
}  
  
ask(  
  "Do you agree?",  
  () => alert("You agreed."),  
  () => alert("You canceled the execution.")  
);  
</script>
```

Nous avons réécrit la fonction ask en utilisant des fonctions fléchées. Si l'utilisateur clique sur OK, le message "You agreed." s'affiche, sinon "You canceled the execution." s'affiche. Cela fonctionne exactement comme l'original mais avec une syntaxe plus concise.



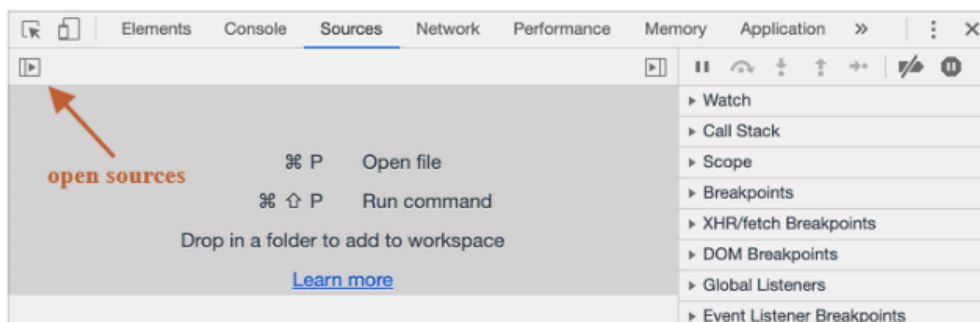
Débogage

Le volet "Sources"

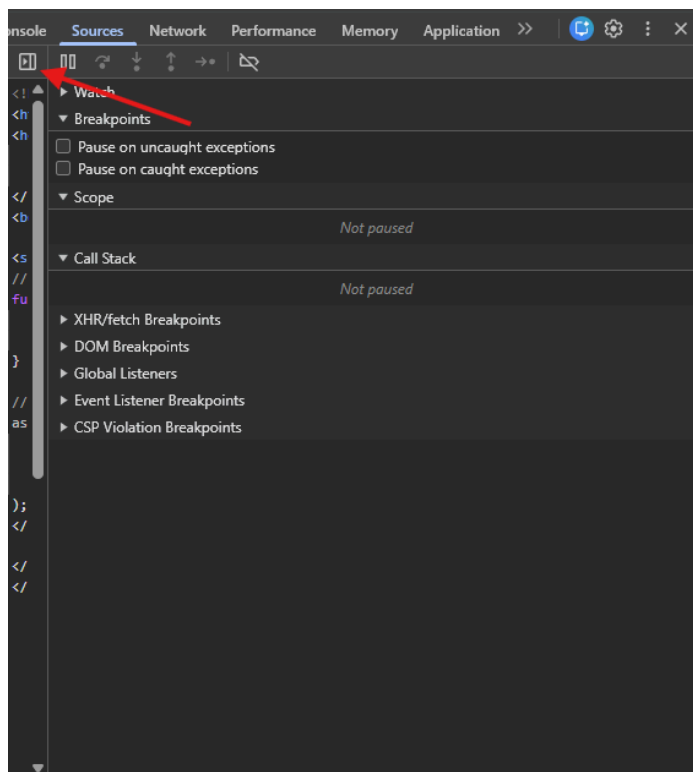
Votre version de Chrome peut sembler un peu différente, mais vous devez tout de même savoir ce qui est là.

- Ouvrez la page d'exemple [index.html](#) dans Chrome.
- Activer les outils de développement avec F12 (Mac: Cmd+Opt+I).
- Sélectionner le volet `Sources`.

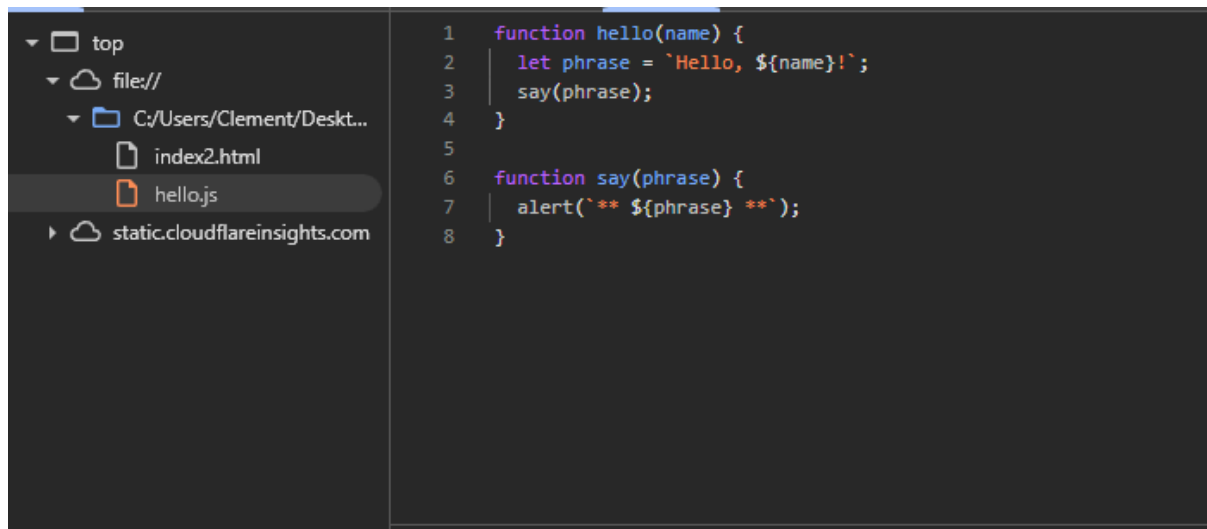
Voici ce que vous devriez voir si vous le faites pour la première fois :



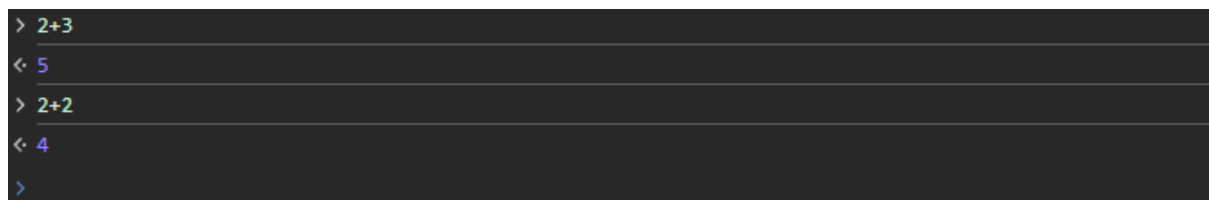
Nous allons ouvrir le volet pour voir les fichiers



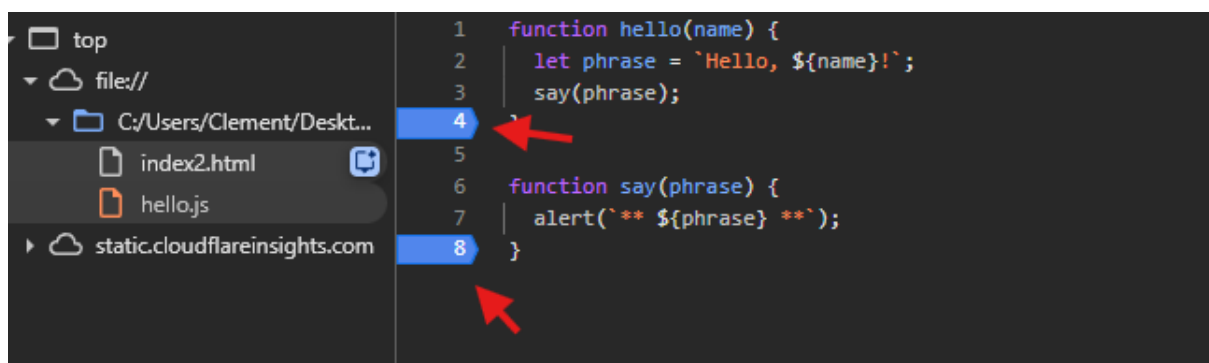
Nous voyons les fichiers ainsi que le code.



En appuyant sur “echap” nous voyons la console apparaître dans laquelle nous pouvons entrer des commandes et voir son résultat.

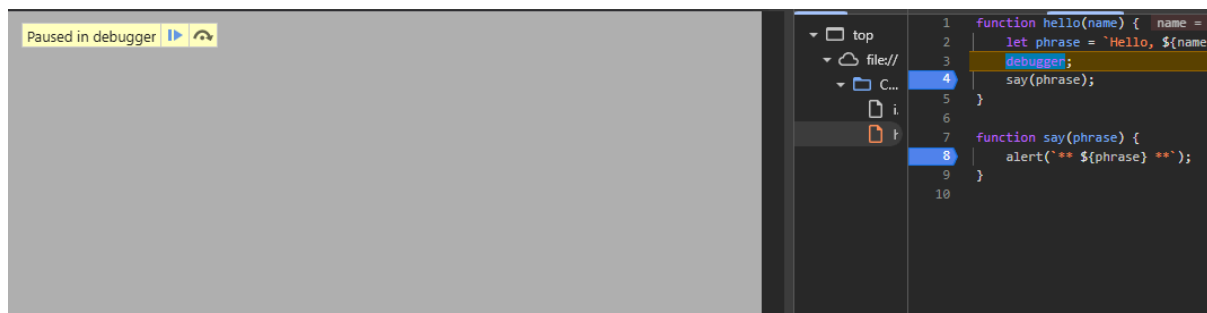


Nous pouvons aussi définir un point d'arrêt en cliquant sur les numéros

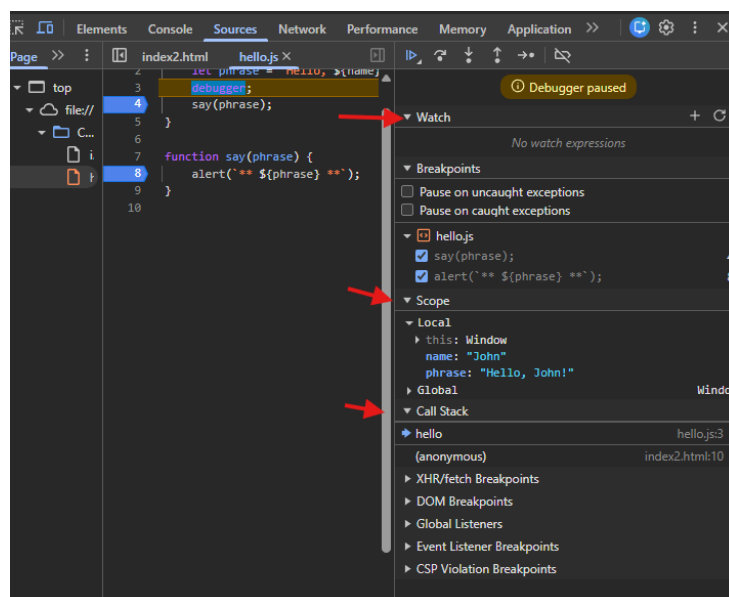


Un breakpoint sert à dire au navigateur de s'arrêter à une ligne précise du code. Quand le code atteint cette ligne, l'exécution se met en pause, ce qui permet de regarder les valeurs des variables et de comprendre le déroulement du programme. Ceci est très utile pour tester le code pas à pas et détecter des erreurs.

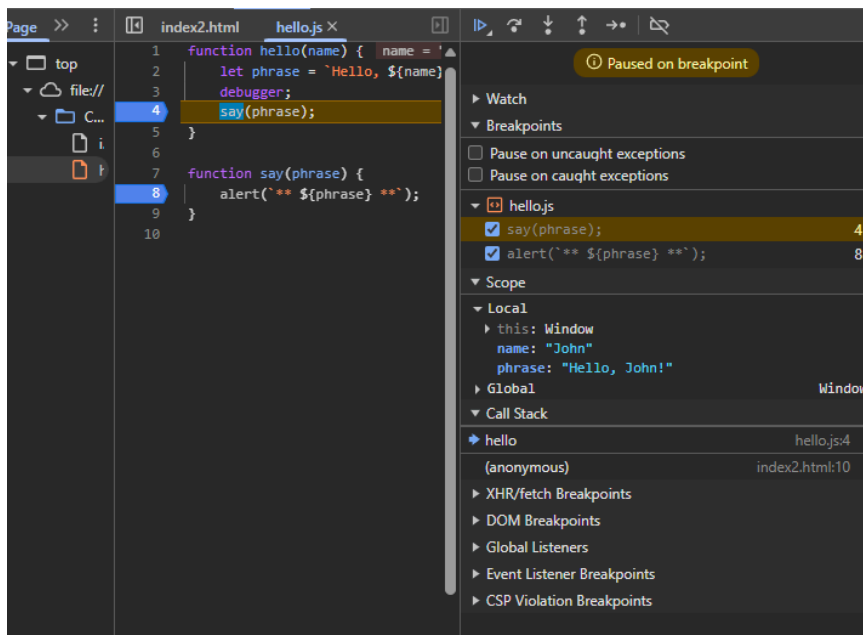
Nous pouvons également suspendre le code en utilisant la commande debugger.



Nous pouvons recharger la page pour activer le débogueur et le code s'est arrêté sur le point d'arrêt dans hello(). Il est possible d'observer les variables locales et globales dans Scope, suivre l'ordre des fonctions appelées dans Call Stack, et utiliser Watch pour surveiller la valeur de n'importe quelle expression pendant l'exécution.



Pour reprendre l'exécution nous pouvons appuyer sur la touche 'F8'



Logging

Pour afficher quelque chose sur la console depuis notre code, utilisez la fonction `console.log`.

Par exemple, cela affiche les valeurs de 0 à 4 sur la console :

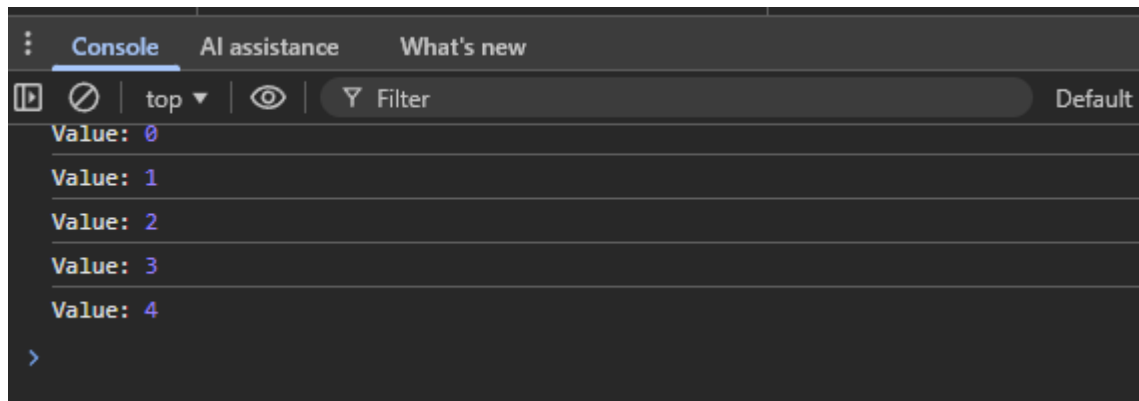
```
1 // ouvrir la console pour visualiser  
2 for (let i = 0; i < 5; i++) {  
3   console.log("value", i);  
4 }
```

Les internautes ne voient pas cette sortie, elle se trouve dans la console. Pour la voir, ouvrez l'onglet Console des outils de développement ou appuyez sur Esc lorsque vous vous trouvez dans un autre onglet : la console en bas s'ouvre.

Si nous avons assez de logging dans notre code, nous pouvons voir ce qui se passe dans les enregistrements, sans le débogueur.

```
<script>

for (let i = 0; i < 5; i++) {
  console.log("Value:", i);
}
</script>
```



Nous avons utilisé une boucle for avec console.log pour afficher les valeurs de 0 à 4 dans la console. Ces messages ne sont visibles que dans les outils de développement et nous permettent de suivre le déroulement du code sans utiliser d'alerte.